

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG**

ĐỀ CƯƠNG ÔN TẬP

Học phần: HỆ THỐNG NHÚNG

**ĐỀ CƯƠNG ÔN TẬP HỌC PHẦN HỆ THỐNG NHÚNG
– THIẾT KẾ HỆ THỐNG NHÚNG DÙNG SoPC TRÊN
NỀN AVALON BUS**

Bốn nội dung ôn tập theo đề cương: khái niệm Master/Bus/Slave, quy trình thiết kế hệ thống nhúng dùng SoPC, bốn project thực hành (PIO HEX, Custom IP HEX, Timer, DMAC) và phân tích hoạt động của Master/Bus/Slave trên Hình 1 – Hình 11.

Thành phố Hồ Chí Minh – Tháng 5 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG
ĐỀ CƯƠNG ÔN TẬP

Học phần: HỆ THỐNG NHÚNG

ĐỀ CƯƠNG ÔN TẬP HỌC PHẦN HỆ THỐNG NHÚNG –
THIẾT KẾ HỆ THỐNG NHÚNG DÙNG SoPC TRÊN NỀN
AVALON BUS

Loại tài liệu	: Đề cương ôn tập theo bốn nội dung của giảng viên
Phạm vi	: Master / Bus / Slave, quy trình SoPC, 4 project thực hành, phân tích Hình 1 – Hình 11
Định dạng	: Bundle Typst — biên dịch bằng <code>typst compile main.typ</code>
Mục đích sử dụng	: Ôn tập trước kỳ thi, đối chiếu lại các nội dung thực hành đã làm

Tài liệu này được biên soạn lại từ đề cương do giảng viên cung cấp, tổ chức nội dung theo trình tự bốn câu hỏi ôn tập. Phần trình bày tập trung vào các điểm cần nhớ khi vấn đáp và cách đọc giản đồ thời gian Avalon Bus, không phải là tài liệu chính thức của môn học. Sinh viên cần đối chiếu lại với slide bài giảng và bộ tài liệu thực hành chính thức trước khi sử dụng.

LỜI NÓI ĐẦU

Tài liệu này được biên soạn theo bốn nội dung ôn tập do giảng viên cung cấp cho học phần HỆ THỐNG NHÚNG. Mục đích của tài liệu là giúp sinh viên ôn tập có hệ thống, từ các khái niệm nền tảng về Master, Bus, Slave trong kiến trúc Avalon, đến quy trình thiết kế một hệ thống nhúng dùng SoPC và các project thực hành cụ thể. Phần cuối tập trung phân tích từng giản đồ thời gian xuất hiện trong đề cương, từ Hình 1 đến Hình 11.

Đề cương gốc được giảng viên đưa ra ở dạng bốn ý lớn: (1) trình bày các khái niệm và chức năng của Master, Bus, Slave; (2) quy trình thiết kế một hệ thống nhúng dùng SoPC; (3) hiểu và thiết kế được bốn project thực hành — Prj1 dùng sáu IP PIO để điều khiển HEX, Prj2 dùng custom IP HEX tự viết, Prj3 dùng Timer cho phần đếm giây, Prj4 dùng DMAC; (4) phân tích hoạt động của Master, Bus và Slave thông qua giản đồ thời gian đính kèm. Tài liệu giữ nguyên thứ tự bốn nội dung này để tiện đối chiếu khi ôn tập.

Tài liệu được trình bày theo hướng cô đọng, hạn chế phân lý thuyết xa đề. Mỗi nội dung đều có phần “Câu hỏi ôn tập” gợi ý ở cuối, giúp sinh viên tự kiểm tra mức độ hiểu bài. Các giản đồ thời gian Avalon được giải thích theo từng cột thời gian (A, B, C, ...) để đọc cùng với hình trong đề cương gốc; phần này không lặp lại hình ảnh mà chỉ mô tả tín hiệu, vì hình đã có sẵn trong đề cương của giảng viên.

Người biên soạn

MỤC LỤC

Lời nói đầu	i
Mục lục	ii
Danh mục chữ viết tắt	vi
Danh sách bảng	vii
1 MASTER, BUS, SLAVE — KHÁI NIỆM VÀ CHỨC NĂNG	1
1.1 Tổng quan kiến trúc Avalon Bus	1
1.2 Master: khái niệm và chức năng	2
1.3 Slave: khái niệm và chức năng	2
1.4 Bus: khái niệm và chức năng	3
1.5 Luồng một giao tác đọc/ghi đầy đủ	4
1.6 Phân biệt giao thức không có và có waitrequest	5
1.7 Vai trò của byteenable, chipselect và readdata trong giao tác	6
1.8 Địa chỉ, độ rộng dữ liệu và byteenable	6
1.9 Quy tắc thiết kế một Avalon-MM Slave tự viết	7
1.10 Mẫu trả lời vấn đáp nhanh	8
1.11 Câu hỏi ôn tập gợi ý	9
2 QUY TRÌNH THIẾT KẾ MỘT HỆ THỐNG NHÚNG DÙNG SoPC	10
2.1 Quan điểm tổng thể	10
2.2 Bước 1 — Khởi tạo project Quartus	10
2.3 Bước 2 — Xây dựng hệ thống trong Qsys (Platform Designer)	11
2.4 Bước 3 — Tích hợp Qsys vào top-level và gán pin	12
2.5 Bước 4 — Tổng hợp và nạp bitstream	12
2.6 Bước 5 — Sinh BSP và viết chương trình phần mềm	13
2.7 Bước 6 — Kiểm thử và gỡ lỗi	13
2.8 Luồng riêng cho bài Custom IP HEX của thầy	14

2.9	Quan hệ giữa file phần cứng và file phần mềm	15
2.10	Checklist trước khi bấm Generate và Compile	16
2.11	Cách debug theo triệu chứng	16
2.12	Biên giới phần cứng - phần mềm trong chương trình C	17
2.13	Tóm tắt quy trình	18
2.14	Câu hỏi ôn tập gợi ý	18
3	BỐN PROJECT THỰC HÀNH	20
3.1	Hướng dẫn ôn theo project	20
3.2	Prj1 — Làm quen với HEX bằng sáu IP PIO và <code>usleep</code>	20
3.2.1	Mục tiêu phần cứng	20
3.2.2	Mã hiển thị và thứ tự các HEX	21
3.2.3	Chi tiết cần nắm thêm cho Prj1	23
3.3	Prj2 — Custom IP HEX điều khiển sáu LED bảy đoạn	24
3.3.1	Mục tiêu phần cứng	24
3.3.2	Mã HDL khái quát của IP	25
3.3.3	Phần mềm	27
3.3.4	Đóng gói IP tự viết trong Component Editor	28
3.3.5	Simulation và lỗi thường gặp trong buổi 4	29
3.3.6	So sánh Prj1 và Prj2	31
3.4	Prj3 — Đồng hồ dùng Timer cho phần delay một giây	31
3.4.1	Mục tiêu phần cứng	31
3.4.2	Phần mềm dùng ISR	32
3.4.3	Các thanh ghi Timer cần nhớ	33
3.4.4	Trình tự cấu hình ngắt Timer	33
3.4.5	So sánh Prj1 và Prj3 về thời gian	34
3.5	Prj4 — Hệ thống có dùng DMAC	34
3.5.1	Mục tiêu phần cứng	34

3.5.2	Trình tự chạy DMA điển hình	35
3.5.3	DMAC vừa là Slave vừa là Master như thế nào?	36
3.5.4	Các điểm cần chú ý khi dùng DMA	37
3.5.5	Cách kiểm chứng DMA đã chạy đúng	37
3.6	So sánh nhanh bốn project	38
3.7	Mẫu trả lời vấn đáp cho từng project	38
3.8	Câu hỏi ôn tập gợi ý	39
4	PHÂN TÍCH HOẠT ĐỘNG CỦA MASTER, BUS, SLAVE TRÊN HÌNH 1 – HÌNH 11	41
4.1	Cách đọc giản đồ thời gian Avalon	41
4.2	Cách nhận dạng nhanh một giản đồ Avalon chưa biết	42
4.3	Bảng ai phát tín hiệu trong giản đồ	43
4.4	Hình 1 — Slave Read cơ bản (zero wait state)	43
4.4.1	Bối cảnh	43
4.4.2	Phân tích từng cột	44
4.5	Hình 2 — Slave Read với 1 chu kỳ chờ (fixed wait state = 1)	44
4.6	Hình 3 — Slave Read với 2 chu kỳ chờ (fixed wait state = 2)	45
4.7	Hình 4 — Slave Read với waitrequest (variable wait state)	45
4.7.1	Đặc điểm	45
4.7.2	Phân tích	45
4.8	Hình 5 — Slave Write cơ bản (zero wait state)	46
4.9	Hình 6 — Slave Write với fixed wait state	47
4.10	Hình 7 — Slave Write với waitrequest	47
4.11	Hình 8 — Master Read nhìn từ phía Master Port (zero wait state)	48
4.12	Hình 9 — Master Read với waitrequest	48
4.13	Hình 10 — Master Write nhìn từ Master Port (zero wait state)	49
4.14	Hình 11 — Master Write với hai chu kỳ waitrequest	50

4.15	Tóm tắt khác biệt giữa các hình	51
4.16	Cặp so sánh hay bị hỏi	52
4.17	Mẫu trả lời khi giảng viên chỉ vào một cột thời gian	52
4.18	Câu hỏi ôn tập gợi ý	53
5	TỔNG KẾT VÀ CHIẾN LƯỢC ÔN TẬP	54
5.1	Bản đồ kiến thức theo bốn nội dung	54
5.2	Lộ trình ôn tập gợi ý	54
5.3	Những lỗi thường gặp khi vấn đáp	55
5.4	Ngân hàng câu hỏi ngắn và ý trả lời	56
5.5	Khung trả lời 2 phút khi bị hỏi một project	58
5.6	Checklist cuối trước khi thi	58
5.7	Lời kết	58
	Tài liệu tham khảo	60

DANH MỤC CHỮ VIẾT TẮT

Chữ viết tắt	Diễn giải
SoPC	System on a Programmable Chip — hệ thống trên một chip khả lập trình
FPGA	Field-Programmable Gate Array — mạch khả trình theo trường
IP	Intellectual Property — khối thiết kế phần cứng đóng gói sẵn
PIO	Parallel Input/Output — IP vào/ra song song
DMA	Direct Memory Access — truy cập bộ nhớ trực tiếp
DMAC	DMA Controller — bộ điều khiển DMA
CPU	Central Processing Unit — bộ xử lý trung tâm (ở đây thường là Nios II)
Avalon-MM	Avalon Memory-Mapped Interface — chuẩn giao tiếp ánh xạ bộ nhớ của Avalon Bus
HDL	Hardware Description Language — ngôn ngữ mô tả phần cứng (Verilog/VHDL)
BSP	Board Support Package — gói hỗ trợ phần cứng cho phần mềm
HAL	Hardware Abstraction Layer — lớp trừu tượng hóa phần cứng
LED	Light-Emitting Diode — đèn LED
SW	Switch — công tắc gạt trên kit
HEX	Bộ hiển thị bảy đoạn (seven-segment display)
IRQ	Interrupt Request — yêu cầu ngắt
ISR	Interrupt Service Routine — chương trình phục vụ ngắt
LSB / MSB	Least / Most Significant Bit — bit có trọng số thấp / cao

Bảng 1 : Danh mục chữ viết tắt sử dụng trong tài liệu

DANH SÁCH BẢNG

Bảng 1	Danh mục chữ viết tắt sử dụng trong tài liệu	vi
Bảng 2	Các chức năng cốt lõi của một Slave Avalon-MM	3
Bảng 3	Chuỗi sự kiện cần nhớ cho giao tác đọc và ghi	5
Bảng 4	Ý nghĩa các tín hiệu chính trong giao thức Avalon-MM	6
Bảng 5	Các quy tắc quan trọng khi viết Slave Avalon-MM	8
Bảng 6	Các chi tiết quan trọng cần kiểm tra trước khi Generate Qsys	12
Bảng 7	Các file quan trọng trong luồng thiết kế SoPC	15
Bảng 8	Checklist lỗi phần cứng thường gặp	16
Bảng 9	Khoanh vùng lỗi theo triệu chứng khi chạy trên kit	17
Bảng 10	Sáu bước thiết kế hệ thống nhúng dùng SoPC	18
Bảng 11	Sơ đồ khối phần cứng cho Prj1	21
Bảng 12	Các điểm thường bị hỏi sâu trong Prj1	23
Bảng 13	Các interface của custom IP HEX trong Component Editor	25
Bảng 14	Các mục cần kiểm tra khi đóng gói IP tự viết	29
Bảng 15	Các lỗi thực hành cần nhớ cho Prj2	30
Bảng 16	So sánh Prj1 và Prj2	31
Bảng 17	Sơ đồ khối phần cứng cho Prj3	31
Bảng 18	Các thanh ghi Timer thường dùng trong Prj3	33
Bảng 19	Ưu nhược điểm của hai cách tạo mốc 1 giây	34
Bảng 20	Sơ đồ khối phần cứng cho Prj4	35
Bảng 21	Hai pha hoạt động của DMAC	36
Bảng 22	Các rủi ro thực tế khi làm Prj4	37
Bảng 23	So sánh nhanh đặc điểm bốn project trong đề cương	38
Bảng 24	Quy trình nhận dạng một giản đồ thời gian Avalon	42
Bảng 25	Nguồn phát của các tín hiệu trong giản đồ Avalon-MM	43

Bảng 26	Tóm tắt nhanh đặc điểm Hình 1 – Hình 11	51
Bảng 27	Các cặp hình nên so sánh được bằng lời	52
Bảng 28	Bản đồ kiến thức tổng quát	54
Bảng 29	Câu hỏi ngắn thường gặp và ý trả lời	57

CHƯƠNG 1

MASTER, BUS, SLAVE — KHÁI NIỆM VÀ CHỨC NĂNG

1.1. Tổng quan kiến trúc Avalon Bus

Trong một hệ thống nhúng được xây dựng theo mô hình SoPC, các khối phần cứng không kết nối trực tiếp với nhau bằng dây dẫn rời, mà giao tiếp với nhau thông qua một hạ tầng truyền thông gọi là **Avalon Bus**. Hạ tầng này được sinh tự động bởi công cụ Qsys (Platform Designer) khi sinh viên kéo các IP vào hệ thống và nối các port với nhau. Trên Avalon Bus, mỗi khối phần cứng đóng một trong hai vai trò: **Master** — khởi tạo các giao tác đọc/ghi, hoặc **Slave** — đáp ứng các giao tác đó. Avalon Bus Module nằm ở giữa, đảm nhiệm việc định tuyến, giải mã địa chỉ, đồng bộ thời gian và xử lý cạnh tranh khi nhiều Master cùng truy cập một Slave.

Mô hình ba thành phần Master – Bus – Slave là cách trừu tượng hóa giúp người thiết kế tách bạch hai mối quan tâm. Một là **điều khiển luồng dữ liệu**: khối nào ra lệnh, khối nào phục vụ, khi nào cần đợi. Hai là **kết nối vật lý**: phần này được công cụ tự lo, người thiết kế chỉ cần khai báo địa chỉ, độ rộng dữ liệu và vài thuộc tính của port. Nhờ tách bạch như vậy, hệ thống có thể được mở rộng bằng cách bổ sung thêm IP mà không phải vẽ lại toàn bộ sơ đồ kết nối.

Theo manual Avalon Bus, cần phân biệt **bus cycle** và **bus transfer**. Bus cycle là một chu kỳ clock của bus, tính từ cạnh lên này đến cạnh lên kế tiếp. Bus transfer là một thao tác đọc hoặc ghi một đối tượng dữ liệu; một transfer có thể chỉ mất một bus cycle, hoặc kéo dài nhiều bus cycle nếu có fixed wait state hoặc `waitrequest`.

Quy ước trong tài liệu

Tài liệu sử dụng quy ước tín hiệu Avalon-MM thường gặp: `address`, `byteenable_n` cho địa chỉ và mặt nạ byte; `read_n`, `write_n` cho yêu cầu đọc/ghi active-low trong manual; `chipselct` để chọn Slave; `readdata`, `writedata` cho dữ liệu hai chiều; `waitrequest` để Slave/Bus yêu cầu Master kéo dài transfer. Trong Platform Designer,

cùng signal type có thể khai báo active-high hoặc active-low. Vì vậy IP HEX buổi 4 dùng `iWrite` active-high, còn manual cũ thường vẽ `write_n` active-low.

1.2. Master: khái niệm và chức năng

Master là khối phần cứng có khả năng **khởi tạo giao tác** trên Avalon Bus. Trong các project thực hành, Master phổ biến nhất là CPU Nios II — chính là khối chạy chương trình C của sinh viên. Khi chương trình C thực hiện một câu lệnh ghi địa chỉ ánh xạ I/O (ví dụ `IOWR_ALTERA_AVALON_PIO_DATA(...)`), Nios II sẽ phát ra một giao tác ghi trên port Master của nó. Ngoài CPU, các khối DMAC và một số IP truyền thông tốc độ cao cũng đóng vai trò Master vì chúng có thể tự sinh ra địa chỉ và yêu cầu truy cập bộ nhớ mà không cần CPU can thiệp từng byte.

Chức năng chính của Master gồm bốn nhóm:

- **Sinh địa chỉ và byteenable:** Master quyết định ô nhớ hoặc thanh ghi nào sẽ bị đọc/ghi, cùng với mặt nạ byte hợp lệ trong word.
- **Phát tín hiệu điều khiển:** Master kích hoạt `read_n` hoặc `write_n` để báo cho Bus biết loại giao tác đang yêu cầu.
- **Cấp/nhận dữ liệu:** với giao tác ghi, Master đặt dữ liệu lên `writedata`; với giao tác đọc, Master nhận `readdata` từ Slave.
- **Tuân thủ tín hiệu đợi:** nếu Slave hoặc Bus phát `waitrequest`, Master phải giữ nguyên các tín hiệu địa chỉ/điều khiển cho đến khi `waitrequest` về 0; đây là điểm phân biệt Master “biết đợi” với một mạch điều khiển đơn giản.

Ghi nhớ. Một port Master không bắt buộc phải là CPU. Khi sinh viên tự viết một IP có khả năng đọc bộ nhớ ngoài (ví dụ một mạch quét bảng tra), IP đó cũng được khai báo là Master trong Qsys. Điểm chung là khả năng **chủ động** sinh ra địa chỉ.

1.3. Slave: khái niệm và chức năng

Slave là khối phần cứng **đáp ứng thụ động** các giao tác từ Master. Trong các project thực hành, Slave bao gồm các IP PIO điều khiển LED, HEX, Switch; các IP Timer; vùng

nhớ on-chip RAM/ROM; các thanh ghi của IP tự viết. Slave không tự khởi tạo giao tác mà chỉ phản ứng khi Bus dẫn tín hiệu đến port của nó.

Trên port Slave, các tín hiệu thường gặp gồm `chipselct`, `address`, `byteenable_n`, `read_n`, `write_n`, `readdata`, `writedata` và tùy chọn `waitrequest`. Slave nhận biết mình được chọn khi `chipselct = 1`; sau đó nó dựa vào `address` để chọn thanh ghi nội bộ, đọc dữ liệu vào (với `write`) hoặc xuất dữ liệu ra (với `read`). Một số Slave đơn giản chỉ cần một chu kỳ để hoàn tất, một số Slave chậm hơn cần nhiều chu kỳ — và đó là lúc các tín hiệu `waitrequest` hoặc số chu kỳ chờ cố định (fixed wait states) được dùng.

Chức năng	Mô tả
Giải mã địa chỉ trong Slave	Khi <code>chipselct = 1</code> , Slave đọc <code>address</code> để chọn đúng thanh ghi nội bộ. Ví dụ thanh ghi giờ ở offset 0, phút ở offset 1, giây ở offset 2.
Phản hồi đọc	Khi <code>read_n = 0</code> và <code>chipselct = 1</code> , Slave đặt giá trị thanh ghi tương ứng lên đường <code>readdata</code> . Master lấy mẫu giá trị này theo thời điểm quy ước trong giao thức.
Tiếp nhận ghi	Khi <code>write_n = 0</code> và <code>chipselct = 1</code> , Slave chốt giá trị <code>writedata</code> vào thanh ghi nội bộ. Một số Slave cho phép chọn byte cần ghi qua <code>byteenable_n</code> .
Phát <code>waitrequest</code>	Nếu Slave chưa kịp xử lý, nó kéo <code>waitrequest</code> lên 1 để Bus thông báo cho Master giữ nguyên giao tác. Khi sẵn sàng, Slave hạ <code>waitrequest</code> xuống 0 và quá trình đọc/ghi hoàn tất.
Sinh ngắt (tùy chọn)	Một Slave có thể có line IRQ để báo sự kiện về bộ điều khiển ngắt; ví dụ Timer hết khoảng đếm hoặc PIO phát hiện cạnh tín hiệu vào.

Bảng 2 : Các chức năng cốt lõi của một Slave Avalon-MM

1.4. Bus: khái niệm và chức năng

Avalon Bus Module không phải là một sợi dây, mà là một **mạng kết nối có cấu trúc** được Qsys sinh ra. Bus đảm nhiệm bốn chức năng chính.

Thứ nhất, **giải mã địa chỉ ở mức hệ thống**: dựa trên dải địa chỉ Master phát ra, Bus xác định Slave nào sẽ được chọn và đặt `chipsselect` của Slave đó. Master không cần biết Slave được gắn ở đâu trên FPGA; nó chỉ cần biết base address mà công cụ đã gán. Manual lưu ý `chipsselect` có thể là tín hiệu tổ hợp sinh từ địa chỉ đã đăng ký, nên không nên thiết kế logic bắt cạnh lên của `chipsselect` hoặc cạnh xuống của `read_n` làm “xung bắt đầu” duy nhất.

Thứ hai, **định tuyến tín hiệu**: Bus chuyển `address`, `byteenable_n`, `read_n`, `write_n`, `writedata` từ Master sang Slave; chuyển `readdata`, `waitrequest` từ Slave về Master. Khi hệ thống có nhiều Master, Bus còn đảm nhiệm việc **trọng tài (arbitration)** để quyết định Master nào được phục vụ trước khi xảy ra xung đột.

Thứ ba, **đồng bộ thời gian**: Bus chèn các tầng pipeline khi cần để đáp ứng yêu cầu về tần số. Người thiết kế nhìn vào giản đồ thời gian thường thấy một vài cột chu kỳ A, B, C, ... có vẻ “thừa”; thực tế đó là độ trễ của Bus và thời gian Slave xử lý.

Thứ tư, **quản lý chu kỳ chờ**: Bus phối hợp giữa số chu kỳ chờ cố định khai báo trong Slave và tín hiệu `waitrequest` động. Nhờ đó Master thấy một giao thức thống nhất, dù Slave chậm hay nhanh.

Vai trò ba thành phần trong một câu

Master ra lệnh, Slave thực hiện, Bus là cầu nối có quyền trọng tài. Khi đọc giản đồ thời gian, hãy xác định trước tín hiệu nào do Master phát, tín hiệu nào do Slave phát; Bus chỉ chuyển tiếp và có thể chèn thêm chu kỳ.

1.5. Luồng một giao tác đọc/ghi đầy đủ

Khi bị hỏi về Master, Bus, Slave, sinh viên không nên chỉ trả lời định nghĩa rời rạc. Cách trả lời an toàn hơn là mô tả trọn vẹn một giao tác đi qua ba thành phần. Một giao tác Avalon-MM luôn có dạng: Master đưa yêu cầu, Bus chọn đúng Slave, Slave phản hồi, sau đó Master kết thúc giao tác. Nếu nhớ được chuỗi này, hầu hết câu hỏi về giản đồ thời gian đều suy ra được.

Bước	Giao tác đọc	Giao tác ghi
1	Master đặt <code>address</code> , <code>byteenable_n</code> và kéo <code>read_n = 0</code> .	Master đặt <code>address</code> , <code>byteenable_n</code> , <code>writedata</code> và kéo <code>write_n = 0</code> .
2	Bus giải mã địa chỉ, chọn Slave tương ứng bằng <code>chipselect = 1</code> .	Bus giải mã địa chỉ và cũng đặt <code>chipselect = 1</code> cho Slave đích.
3	Slave đọc địa chỉ nội bộ, chuẩn bị dữ liệu và đặt lên <code>readdata</code> .	Slave kiểm tra <code>write_n</code> , <code>chipselect</code> , <code>byteenable_n</code> rồi chốt <code>writedata</code> vào thanh ghi.
4	Nếu chưa có dữ liệu, Slave giữ <code>waitrequest = 1</code> ; Master phải giữ nguyên yêu cầu.	Nếu chưa ghi được, Slave giữ <code>waitrequest = 1</code> ; Master phải giữ nguyên dữ liệu ghi.
5	Khi <code>waitrequest = 0</code> , Master lấy mẫu <code>readdata</code> và kết thúc đọc.	Khi <code>waitrequest = 0</code> , Slave đã nhận dữ liệu; Master rút <code>write_n</code> và kết thúc ghi.

Bảng 3 : Chuỗi sự kiện cần nhớ cho giao tác đọc và ghi

Ghi nhớ. Điểm mấu chốt: trong Read, dữ liệu quan trọng đi từ Slave về Master qua `readdata` ; trong Write, dữ liệu quan trọng đi từ Master sang Slave qua `writedata` . Các tín hiệu còn lại chủ yếu giúp hai bên thống nhất “đang truy cập thanh ghi nào, vào lúc nào, có phải đợi không”.

1.6. Phân biệt giao thức không có và có `waitrequest`

Avalon hỗ trợ hai cách quản lý độ trễ của Slave:

- **Fixed wait states:** số chu kỳ chờ được khai báo cố định khi cấu hình Slave trong Qsys. Bus tự động kéo dài giao tác đúng số chu kỳ đó. Cách này đơn giản, phù hợp với Slave có độ trễ ổn định (PIO, thanh ghi cấu hình).
- **Variable wait states với `waitrequest`** : Slave tự kéo `waitrequest` lên/xuống tùy trạng thái. Cách này phù hợp với Slave có thời gian xử lý thay đổi (FIFO, bộ nhớ ngoài, IP truyền thông).

Sự khác biệt giữa hai cách này thể hiện rất rõ ở các giản đồ thời gian Hình 1 – Hình 11 trong đề cương. Hình 1, Hình 5, Hình 8 và Hình 10 minh họa transfer cơ bản không có wait state; Hình 2, Hình 3 và Hình 6 minh họa wait states cố định; Hình 4, Hình 7, Hình 9 và Hình 11 minh họa transfer bị kéo dài bởi `waitrequest`.

1.7. Vai trò của `byteenable`, `chipselct` và `readdata` trong giao tác

Ba tín hiệu thường bị nhầm lẫn khi đọc giản đồ là `byteenable_n`, `chipselct` và `readdata`. Tài liệu tách riêng để sinh viên dễ phân biệt.

Tín hiệu	Diễn giải
<code>address</code> , <code>byteenable_n</code>	Master phát ra cùng thời điểm để xác định word và byte cần truy cập. <code>byteenable_n[i] = 0</code> nghĩa là byte thứ <i>i</i> trong word có hiệu lực.
<code>chipselct</code>	Bus phát ra dành riêng cho Slave được chọn; chỉ một Slave nhận <code>chipselct = 1</code> tại một thời điểm. Slave dựa vào tín hiệu này để biết mình đang được giao tiếp.
<code>read_n</code>	Master phát ra để báo giao tác là đọc. Active-low: 0 nghĩa là đang yêu cầu đọc.
<code>write_n</code>	Tương tự <code>read_n</code> nhưng cho ghi.
<code>writedata</code>	Dữ liệu Master đặt lên Bus khi ghi. Có hiệu lực kèm với <code>write_n</code> và <code>chipselct</code> .
<code>readdata</code>	Dữ liệu Slave xuất ra khi đọc. Master lấy mẫu vào cuối chu kỳ đọc theo quy ước Avalon.
<code>waitrequest</code>	Slave (hoặc Bus) phát ra để giữ giao tác lại. Khi <code>waitrequest = 1</code> , Master phải giữ nguyên <code>address</code> , <code>read_n</code> , <code>write_n</code> , <code>writedata</code> .

Bảng 4 : Ý nghĩa các tín hiệu chính trong giao thức Avalon-MM

1.8. Địa chỉ, độ rộng dữ liệu và `byteenable`

Trong các bài tự viết IP, giảng viên rất hay hỏi vì sao IP chỉ cần điều khiển một LED bảy đoạn 7 bit hoặc một chữ số 0-9 nhưng port dữ liệu lại là `writedata[31:0]`. Lý do là Avalon-MM được tổ chức theo độ rộng dữ liệu của Master/Bus để phần mềm truy cập thuận tiện, còn bên trong Slave có thể chỉ dùng một vài bit có nghĩa.

Ví dụ IP HEX buổi 4 khai báo `iWriteData[31:0]`, nhưng để hiển thị một chữ số thì HDL chỉ cần `iWriteData[3:0]`. Nếu phần mềm gọi `IOWR(HEX_0_BASE, 3, 8)`, CPU phát một transfer ghi đến base address của IP; Bus chọn Slave bằng `chipselct`; bên trong IP, `iAddress = 3` chọn `oHex3`, còn `iWriteData[3:0] = 8` được giải mã ra mã LED bảy đoạn cho số 8.

Tín hiệu `byteenable_n[3:0]` cho biết byte nào trong word được phép ghi. Vì hậu tố `_n` nghĩa là active-low, `byteenable_n = 4'b1110` nghĩa là byte thấp nhất có hiệu lực, còn ba byte cao bị che. Với các IP register/control đơn giản như HEX, nhiều bài lab có thể không đưa `byteenable_n` ra HDL; khi đó IP mặc định coi lệnh ghi là ghi cả word và tự dùng các bit cần thiết.

Địa chỉ cũng cần phân biệt hai mức:

- **Base address:** địa chỉ bắt đầu của cả IP trong không gian địa chỉ hệ thống, do Qsys gán. Phần mềm thấy địa chỉ này qua macro như `MY_IP_BASE`.
- **Offset/địa chỉ nội bộ:** địa chỉ con bên trong IP. Với custom IP HEX, index 0 đến 5 trong `IOWR(HEX_0_BASE, index, digit)` được đưa vào `iAddress[2:0]` để chọn `HEX0` đến `HEX5`.

Cách giải thích khi bị hỏi về offset

CPU không tự đưa trực tiếp “thanh ghi giờ” cho IP. CPU chỉ phát một địa chỉ tuyệt đối. Bus nhìn địa chỉ tuyệt đối đó, chọn đúng IP bằng `chipselct`, rồi đưa phần offset cho IP. IP dùng offset để chọn thanh ghi nội bộ. Vì vậy base address thuộc về hệ thống, còn offset thuộc về thiết kế bên trong Slave.

1.9. Quy tắc thiết kế một Avalon-MM Slave tự viết

Một IP tự viết đóng vai trò Slave cần tối thiểu ba phần: phần giao tiếp Bus, phần thanh ghi nội bộ, và phần logic chức năng. Nếu trộn ba phần này quá lẫn lộn, mạch vẫn có thể chạy nhưng rất khó debug khi bị sai địa chỉ hoặc sai thời điểm đọc/ghi.

Quy tắc	Nên làm	Lỗi hay gặp
Reset rõ ràng	Đặt các thanh ghi về giá trị xác định khi <code>reset_n = 0</code> .	Không reset khiến giờ/phút/giây ban đầu là X khi mô phỏng hoặc giá trị ngẫu nhiên trên kit.
Ghi đồng bộ	Chốt <code>writedata</code> ở cạnh lên clock khi <code>chipselect && !write_n</code> .	Ghi trong khối tổ hợp làm sinh latch hoặc ghi sai nhiều lần trong một giao tác.
Đọc ổn định	Xuất <code>readdata</code> theo <code>address</code> ; nếu cần trễ thì dùng wait state hoặc <code>waitrequest</code> .	Đổi <code>readdata</code> quá muộn khiến Master lấy mẫu sai dữ liệu.
Không ghi vào read-only	Thanh ghi chỉ đọc như Switch chỉ phản ánh input ngoài.	Cho CPU ghi vào Switch register làm mô hình chức năng không đúng với phần cứng thật.
Xử lý giá trị vượt ngưỡng	Chặn giờ > 23, phút/giây > 59 hoặc tự modulo về khoảng hợp lệ.	Cho phép ghi 99 giây rồi carry bị sai khi chạy đồng hồ.

Bảng 5 : Các quy tắc quan trọng khi viết Slave Avalon-MM

1.10. Mẫu trả lời vấn đáp nhanh

Nếu câu hỏi chỉ là “Master, Bus, Slave là gì?”, có thể trả lời theo mẫu sau để đủ ý:

Mẫu trả lời

Trong hệ thống SoPC dùng Avalon-MM, Master là khối chủ động phát giao tác đọc/ghi, ví dụ CPU Nios II hoặc DMAC. Slave là khối bị truy cập, ví dụ PIO, Timer, on-chip memory hoặc IP tự viết; nó phản hồi thông qua `readdata` hoặc nhận `writedata`. Avalon Bus nằm giữa để giải mã địa chỉ, phát `chipselect`, định tuyến tín hiệu, xử lý arbitration nếu có nhiều Master và chèn chu kỳ chờ thông qua fixed wait state hoặc `waitrequest`.

1.11. Câu hỏi ôn tập gợi ý

Ghi nhớ. Sinh viên nên trả lời được các câu hỏi sau bằng lời, không cần nhìn lại tài liệu:

1. Phân biệt vai trò của Master và Slave; lấy hai ví dụ Master và hai ví dụ Slave trong các project đã làm.
2. Avalon Bus Module thực hiện những chức năng gì? Tại sao không thể nối trực tiếp Master với Slave?
3. Giải thích tín hiệu `chipsselect` được sinh ra ở đâu, ai dùng và có vai trò gì.
4. Khi nào Slave dùng fixed wait states, khi nào dùng `waitrequest` ?
5. Trong giao tác đọc, Slave đặt dữ liệu lên `readdata` ở thời điểm nào, Master lấy mẫu vào thời điểm nào?
6. Giải thích sự khác nhau giữa base address của IP và offset thanh ghi bên trong IP.
7. Nếu tự viết một Slave mà quên xét `chipsselect` , lỗi gì có thể xảy ra?

CHƯƠNG 2

QUY TRÌNH THIẾT KẾ MỘT HỆ THỐNG NHÚNG DÙNG SoPC

2.1. Quan điểm tổng thể

Một hệ thống nhúng dùng SoPC luôn được phân thành hai nửa song song: nửa **phần cứng** được tổng hợp xuống FPGA dưới dạng cấu hình bitstream, và nửa **phần mềm** chạy trên CPU mềm (Nios II) bên trong FPGA. Hai nửa này gặp nhau ở “biên giới” là không gian địa chỉ ánh xạ I/O: phần cứng quy định Slave nào nằm ở base address nào, phần mềm sử dụng đúng các base address đó để đọc/ghi thanh ghi.

Vì vậy, quy trình thiết kế cũng có hai luồng. Luồng phần cứng dùng Quartus Prime kết hợp với Qsys (Platform Designer) để xếp đặt CPU, IP và bộ nhớ. Luồng phần mềm dùng Nios II Software Build Tools (Eclipse) để viết chương trình C, build BSP từ hệ thống Qsys vừa sinh, và nạp lên CPU đã được tổng hợp. Khi sinh viên thay đổi hệ thống Qsys, BSP phải được sinh lại để các macro địa chỉ phần mềm khớp với phần cứng.

Nguyên tắc bám đề cương

Đề cương yêu cầu **nắm rõ ràng cách thiết kế phần cứng và chương trình phần mềm**.

Vì vậy phần ôn tập sau đây trình bày song song hai luồng, và mỗi project ở chương sau cũng sẽ được mô tả theo cùng cấu trúc: phần cứng — phần mềm.

2.2. Bước 1 — Khởi tạo project Quartus

Project Quartus là vỏ bọc ngoài cùng. Sinh viên chọn họ FPGA, đời chip, gói chân và đặt thư mục project. Tại bước này, Quartus chưa biết gì về Avalon Bus; nó chỉ biết một top-level entity sẽ được sinh ra. Vai trò chính của Quartus là **tổng hợp toàn bộ hệ thống** sau khi Qsys đã sinh ra IP, gắn các pin của top-level vào chân FPGA, và nạp file `.sof` xuống kit.

2.3. Bước 2 — Xây dựng hệ thống trong Qsys (Platform Designer)

Đây là bước quan trọng nhất ở phía phần cứng. Trong Qsys, sinh viên thực hiện các thao tác sau:

- **Thêm CPU:** chọn Nios II/e (economy), Nios II/s (small) hoặc Nios II/f (fast) tùy yêu cầu. Cấu hình kích thước instruction cache, data cache, vị trí reset vector và exception vector — hai vector này phải nằm trong vùng nhớ hợp lệ (on-chip memory hoặc SDRAM).
- **Thêm bộ nhớ:** thường là on-chip memory để chứa code và stack cho các bài tập đơn giản; nếu chương trình lớn thì dùng SDRAM controller.
- **Thêm các IP I/O:** PIO cho LED/Switch/HEX; Timer cho đếm thời gian; UART/JTAG UART cho gỡ lỗi; DMAC cho truyền dữ liệu khối. Với bài HEX buổi 3, sáu LED bảy đoạn có thể được tạo bằng sáu PIO output riêng `HEX0` đến `HEX5`.
- **Thêm IP tự viết (nếu cần):** được đóng gói qua Component Editor với tệp HDL và mô tả interface Avalon-MM Slave. Với bài HEX buổi 4, IP `HEX` có `chipselect`, `address[2:0]`, `write`, `writedata[31:0]` và sáu output conduit `hex0` ... `hex5`.
- **Kết nối Master – Slave:** nhấp vào lưới Connections để nối port Master của CPU với port Slave của các IP. Kết nối DMAC (vốn vừa là Master vừa là Slave) cần đặc biệt cẩn thận.
- **Gán base address:** dùng “Assign Base Addresses” để Qsys tự gán, hoặc tự nhập tay nếu cần địa chỉ cố định cho phần mềm có sẵn.
- **Gán IRQ:** với các IP có ngắt (Timer, UART), số IRQ phải duy nhất trên cùng một CPU.
- **Generate:** Qsys sinh ra một module Verilog/VHDL chứa toàn bộ hệ thống và đường Bus, kèm tệp `.qsys` và thư mục con chứa tất cả IP.

Mục	Lưu ý khi thi
Reset vector	Trở vào on-chip memory hoặc SDRAM đã có chứa code; nếu trở sai, CPU không khởi động.
Exception vector	Phải nằm trong vùng nhớ hợp lệ; thường để cùng vùng với reset vector.
Base address	Mỗi Slave phải có dải địa chỉ riêng, không trùng. Sau khi gán xong, ghi nhớ để dùng trong phần mềm thông qua macro <code>_BASE</code> của BSP.
IRQ number	Đặt sao cho mức ưu tiên hợp lý: thiết bị thời gian (Timer) thường có IRQ thấp, các IP cần đáp ứng nhanh để IRQ thấp hơn.
Clock domain	Hệ thống đơn giản dùng một clock duy nhất; nếu có IP chạy clock khác, phải khai báo clock crossing.

Bảng 6 : Các chi tiết quan trọng cần kiểm tra trước khi Generate Qsys

2.4. Bước 3 — Tích hợp Qsys vào top-level và gán pin

Sau khi Qsys generate, sinh viên trở về Quartus và tạo một top-level (file `.v` / `.vhd` hoặc Block Diagram). Top-level này thực chất chỉ làm một việc: instantiate module Qsys vừa sinh, nối các port external (LED, Switch, HEX, clock, reset) ra chân vật lý. Bước gán pin được thực hiện qua Pin Planner và phụ thuộc vào sơ đồ kit thực tế (DE2-115, DE10-Lite, DE0-Nano, ...). Sai pin là lỗi phổ biến và rất khó nhìn ra trên kit, vì hệ thống vẫn chạy bên trong nhưng không có hiển thị.

2.5. Bước 4 — Tổng hợp và nạp bitstream

Quartus thực hiện Analysis & Synthesis, Fitter, Assembler, Timing Analyzer, sinh ra `.sof`. Sinh viên dùng Programmer để nạp `.sof` xuống FPGA qua JTAG. Khi cần lưu cấu hình vào EPCS để chạy không cần JTAG, có thể chuyển sang `.jic`. Trong phạm vi học phần, hầu hết bài thực hành chỉ cần nạp `.sof`.

2.6. Bước 5 — Sinh BSP và viết chương trình phần mềm

Phía phần mềm dùng Nios II Software Build Tools for Eclipse (hoặc command-line).

Quy trình điển hình:

1. Tạo Application và BSP from Template, chọn template “Hello World Small” hoặc “Hello World” tùy IP cần.
2. Trong BSP Editor, tắt các tính năng không cần thiết để giảm dung lượng (newlib reduced, không dùng C++, không dùng floating point).
3. Sinh BSP — Eclipse sinh `system.h` chứa các macro `*_BASE`, `*_IRQ`, `*_DATA_WIDTH`. Đây là cầu nối duy nhất giữa phần mềm và phần cứng.
4. Viết file C, dùng các macro `IORD_*` và `IOWR_*` của HAL hoặc gọi trực tiếp `*((volatile int*)BASE) = ...`.
5. Build, nạp xuống CPU bằng “Run As → Nios II Hardware”.

```
1 // Ví dụ ghi/đọc thanh ghi PIO trong chương trình C
2 #include "system.h"
3 #include "altera_avalon_pio_regs.h"
4
5 int main(void) {
6     unsigned int sw_value = IORD_ALTERA_AVALON_PIO_DATA(SW_BASE);
7     IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, sw_value);
8     return 0;
9 }
```

Ghi nhớ. Bất kỳ thay đổi nào của Qsys đều cần **sinh lại BSP** và **build lại Application**. Đây là lỗi rất hay gặp khi sinh viên đổi base address rồi quên sinh BSP, dẫn đến chương trình ghi vào địa chỉ sai và “không thấy gì xảy ra trên kit”.

2.7. Bước 6 — Kiểm thử và gỡ lỗi

Việc kiểm thử nên đi từ đơn giản đến phức tạp. Trước hết, kiểm tra xem CPU đã chạy chưa bằng cách `printf` ra JTAG UART. Sau đó kiểm tra từng IP riêng lẻ: LED có

sáng đúng giá trị không, Switch có đọc đúng không, HEX có hiển thị đúng không. Cuối cùng mới kết hợp toàn bộ logic. Khi gặp lỗi, có ba điểm cần soi:

- **Phần cứng Qsys:** kết nối Master – Slave có đúng không, IRQ có trùng không.
- **Pin assignment:** chân external có nối đúng vào kit không.
- **Phần mềm:** có dùng đúng macro `_BASE` từ `system.h` không, có quên sinh lại BSP không.

2.8. Luồng riêng cho bài Custom IP HEX của thầy

File `Custome_IP_hex.pdf` nhấn mạnh một chuỗi thao tác thực hành rất cụ thể cho buổi 4. Khi vấn đáp, sinh viên nên kể được luồng này theo đúng thứ tự thay vì chỉ nói chung chung “tạo IP rồi chạy”:

1. Viết module Verilog `HEX` với các port `iClk`, `iRst_n`, `iChipSelect`, `iAddress[2:0]`, `iWrite`, `iWriteData[31:0]`, `oHex0` ... `oHex5`.
2. Mở Component Editor, thêm file HDL, ánh xạ signal type: `chipselect`, `address`, `write`, `writedata`, `clk`, `reset_n`, và sáu output `conduit_end`.
3. Thêm component `hex_0` vào Platform Designer, nối `clock_sink`, `reset_sink`, nối Avalon-MM Slave của `hex_0` với data master của Nios II, export conduit `hex`.
4. Generate HDL, sau đó chọn **Generate Testbench System** nếu cần mô phỏng.
5. Sinh BSP lại để `system.h` có macro `HEX_0_BASE`, rồi viết C dùng `IOWR(HEX_0_BASE, index, digit)`.
6. Nếu chạy mô phỏng, thêm các tín hiệu vào waveform và dùng lệnh `run 500 ms` để quan sát.

Điểm phân biệt buổi 3 và buổi 4

Buổi 3 dùng sáu PIO có sẵn, phần mềm tự ghi mã đoạn active-low cho từng HEX. Buổi 4 dùng một custom IP duy nhất, phần mềm ghi chữ số 0 đến 9 vào offset 0 đến 5, còn HDL trong IP tự giải mã ra sáu ngõ `oHex`.

2.9. Quan hệ giữa file phần cứng và file phần mềm

Một điểm dễ mất điểm khi vấn đáp là không nói được file nào sinh ra từ bước nào. Trong SoPC, mỗi file có vai trò riêng, không nên gom chung thành “Quartus sinh hết”.

Thành phần	Sinh/tạo ở đâu	Vai trò khi chạy hệ thống
<code>.qsys</code>	Platform Designer	Lưu cấu hình hệ thống: CPU, memory, PIO, Timer, DMAC, địa chỉ, IRQ, kết nối Master-Slave.
Module Qsys <code>.v</code> / <code>.vhd</code>	Generate HDL trong Qsys	Là khối phần cứng tổng hợp xuống FPGA, chứa Avalon interconnect và các IP.
Top-level HDL	Người thiết kế viết hoặc dùng schematic	Instantiate hệ thống Qsys và nối port external ra clock, reset, LED, SW, HEX.
<code>.qsf</code>	Quartus project	Chứa pin assignment, clock constraint, file source được thêm vào project.
<code>.sof</code>	Quartus Assembler	Bitstream nạp xuống FPGA qua JTAG. Nếu thiếu bước này, phần cứng mới chưa tồn tại trên kit.
<code>system.h</code>	BSP Generator	Chứa macro địa chỉ và IRQ để chương trình C truy cập đúng phần cứng.
<code>.elf</code>	Nios II SBT/ Eclipse	Chương trình phần mềm nạp vào CPU Nios II sau khi FPGA đã có phần cứng tương ứng.

Bảng 7 : Các file quan trọng trong luồng thiết kế SoPC

Câu nói an toàn khi mô tả luồng build

Luồng đúng là: thiết kế Qsys và top-level trước, compile Quartus để nạp `.sof`, sau đó sinh BSP từ phần cứng mới, build application để có `.elf`, cuối cùng nạp `.elf` lên CPU Nios II. Nếu đổi Qsys nhưng chỉ build lại C thì chương trình vẫn đang chạy trên phần cứng hoặc địa chỉ cũ.

2.10. Checklist trước khi bấm Generate và Compile

Trước khi Generate Qsys, nên kiểm tra theo danh sách sau. Đây là các lỗi nhỏ nhưng thường làm mất rất nhiều thời gian trong phòng lab.

Mục kiểm tra	Câu hỏi tự kiểm	Hậu quả nếu sai
Clock/reset	Tất cả IP có nối cùng clock và reset phù hợp chưa? Reset active-high hay active-low?	CPU không chạy, Timer không đếm, IP tự viết giữ trạng thái sai.
Memory cho CPU	Instruction master và data master của Nios II có nối đến bộ nhớ chứa code chưa?	Nạp <code>.elf</code> được nhưng CPU không fetch lệnh đúng.
Reset/exception vector	Hai vector có trỏ vào on-chip memory hoặc SDRAM hợp lệ không?	CPU treo ngay sau reset hoặc không xử lý được ngắt.
Base address	Các Slave có dải địa chỉ không trùng nhau không?	CPU ghi một IP nhưng IP khác bị chọn, kết quả quan sát sai.
IRQ	Mỗi IP phát ngắt có số IRQ riêng và đã nối vào interrupt receiver của CPU chưa?	ISR không bao giờ chạy hoặc chạy sai nguồn ngắt.
Export port	PIO/Custom IP cần ra chân kit đã export chưa? Tên port có rõ ràng không?	Compile được nhưng top-level không có tín hiệu để nối ra LED/SW/HEX.
Pin Planner	Tên port top-level có đúng với bảng chân kit không?	Mạch bên trong đúng nhưng hiển thị ngoài kit không đúng.

Bảng 8 : Checklist lỗi phần cứng thường gặp

2.11. Cách debug theo triệu chứng

Khi hệ thống không chạy, nên debug theo triệu chứng quan sát được thay vì sửa mò. Bảng sau giúp khoanh vùng nhanh.

Triệu chứng	Nguyên nhân có khả năng cao	Cách kiểm tra
Không in được <code>printf</code>	JTAG UART chưa thêm, CPU chưa chạy, reset vector sai, chưa nạp <code>.sof</code> .	Nạp lại <code>.sof</code> , kiểm tra JTAG UART trong Qsys, thử template Hello World.
LED/HEX không đổi	Sai base address, quên sinh BSP, sai pin, PIO chưa export.	Mở <code>system.h</code> xem macro <code>_BASE</code> , dùng <code>printf</code> in giá trị đang ghi, kiểm tra Pin Planner.
Đọc Switch luôn bằng 0	PIO input chưa nối port ngoài, sai hướng PIO, sai pin.	Kiểm tra cấu hình PIO là input, port đã export và nối trong top-level.
Timer ngắt liên tục	Quên xóa cờ timeout trong ISR hoặc period quá nhỏ.	Trong ISR ghi 0 vào <code>STATUS</code> , in biến đếm ngắt để xem tần suất.
ISR không chạy	Chưa enable interrupt trong Timer, chưa đăng ký ISR, IRQ chưa nối hoặc sai macro IRQ.	Kiểm tra control register, <code>alt_ic_isr_register</code> , <code>TIMER_IRQ</code> trong <code>system.h</code> .
DMA không xong	Sai địa chỉ nguồn/đích, sai length, chưa prepare kênh RX trước TX, vùng nhớ không cho DMAC truy cập.	In địa chỉ buffer, kiểm tra alignment, thử length nhỏ trước.

Bảng 9 : Khoanh vùng lỗi theo triệu chứng khi chạy trên kit

2.12. Biên giới phần cứng - phần mềm trong chương trình C

Trong phần mềm C, các macro trong `system.h` không phải là “biến” bình thường mà là địa chỉ phần cứng. Khi gọi `IOWR_32DIRECT(BASE, OFFSET, DATA)`, CPU phát một giao tác ghi trên Avalon Bus. Khi gọi `IORD_32DIRECT(BASE, OFFSET)`, CPU phát một giao tác đọc và đợi dữ liệu trả về. Từ góc nhìn C, đó chỉ là một dòng lệnh; từ góc nhìn phần cứng, đó là đầy đủ chuỗi Master - Bus - Slave.

```
1 // Hai cách truy cập cùng một thanh ghi Avalon-MM
2 IOWR_32DIRECT(MY_IP_BASE, 0x0, 12); // Ghi thanh ghi offset 0
```

```

3 int value = IORD_32DIRECT(MY_IP_BASE, 0x4); // Đọc thanh ghi offset 4
4
5 // Dạng con trỏ volatile tương đương
6 volatile unsigned int *reg0 = (volatile unsigned int *) (MY_IP_BASE +
7 0x0);
8 *reg0 = 12;

```

Ghi nhớ. Từ khóa `volatile` rất quan trọng khi truy cập thanh ghi phần cứng bằng con trỏ. Nếu không có `volatile`, compiler có thể tối ưu bỏ lần đọc/ghi vì tưởng đó là vùng nhớ bình thường, trong khi thực tế mỗi lần đọc/ghi đều tạo giao tác trên Bus.

2.13. Tóm tắt quy trình

Bước	Tên	Đầu ra
1	Khởi tạo Quartus project	File <code>.qpf</code> cùng cấu hình họ FPGA.
2	Thiết kế hệ thống Qsys	Module hệ thống Avalon, file <code>.qsys</code> , danh sách Slave và base address.
3	Top-level và gán pin	Top-level instantiate hệ thống Qsys; bảng pin assignment khớp kit.
4	Tổng hợp và nạp	File <code>.sof</code> được nạp vào FPGA qua JTAG.
5	Sinh BSP và viết phần mềm	<code>system.h</code> , ứng dụng C đã build, file <code>.elf</code> chạy trên Nios II.
6	Kiểm thử và gỡ lỗi	Hệ thống chạy đúng yêu cầu, kết quả quan sát được trên kit.

Bảng 10 : Sáu bước thiết kế hệ thống nhúng dùng SoPC

2.14. Câu hỏi ôn tập gợi ý

Ghi nhớ.

1. Liệt kê sáu bước thiết kế hệ thống nhúng dùng SoPC. Bước nào là bước “biên giới” giữa phần cứng và phần mềm?
2. Vai trò của Qsys (Platform Designer) là gì? Khi nào cần Generate lại Qsys?

3. Tập `system.h` được sinh ra từ đâu, chứa thông tin gì, dùng làm gì trong phần mềm?
4. Khi đổi base address của một Slave trong Qsys, cần làm gì ở phía phần mềm để chương trình hoạt động trở lại?
5. Tại sao reset vector và exception vector phải trỏ vào vùng nhớ hợp lệ?

CHƯƠNG 3

BỐN PROJECT THỰC HÀNH

3.1. Hướng dẫn ôn theo project

Bốn project trong đề cương đều có chung một bài toán “đồng hồ giờ – phút – giây” hoặc biến thể, nhưng mỗi project tập trung vào một khía cạnh khác nhau. Sự khác biệt nằm ở cách giải quyết hai bài toán phụ: (1) đếm thời gian một giây, và (2) tổ chức thanh ghi điều khiển giờ/phút/giây. Cách phân chia này giúp người học so sánh dễ dàng các kỹ thuật: dùng IP có sẵn so với tự viết IP, dùng vòng lặp CPU so với dùng Timer, dùng truyền byte/byte so với dùng DMAC.

Khung trả lời cho mỗi project

Khi vấn đáp một project, sinh viên nên trình bày theo bốn mục cố định: (1) sơ đồ khối phần cứng trong Qsys; (2) bảng địa chỉ và mục đích từng Slave; (3) thuật toán cập nhật giờ/phút/giây; (4) ý chính của chương trình C. Khung này áp dụng đồng nhất cho cả bốn project, chỉ thay đổi chi tiết bên trong.

3.2. Prj1 — Làm quen với HEX bằng sáu IP PIO và `usleep`

3.2.1 Mục tiêu phần cứng

Project đầu tiên trong slide của giảng viên là bài “làm quen với HEX trên DE10 Standard”. Phần cứng chỉ dùng IP có sẵn trong Platform Designer: CPU Nios II, on-chip memory, JTAG UART và sáu khối PIO output tương ứng `HEX0` đến `HEX5`. Mỗi PIO là một Avalon-MM Slave riêng; CPU Nios II là Master duy nhất, phát lệnh ghi đến từng base address `HEX0_BASE`, `HEX1_BASE`, ..., `HEX5_BASE`.

Trên DE10 Standard, mỗi LED bảy đoạn dùng mã active-low. Vì vậy số 0 không ghi bằng giá trị nhị phân 0, mà ghi bằng mã đoạn tương ứng. Slide của thầy dùng bảng mã 8 bit gồm cả bit dấu chấm thập phân: 0 là `0xC0`, 1 là `0xF9`, 2 là `0xA4`, 3 là `0xB0`, 4 là `0x99`, 5 là `0x92`, 6 là `0x82`, 7 là `0xF8`, 8 là `0x80`, 9 là `0x90`.

IP	Vai trò	Mục đích
Nios II	Master	Chạy chương trình C, gọi <code>IOWR</code> để ghi mã LED bảy đoạn đến từng PIO.
On-chip memory	Slave	Chứa code, stack và data; reset vector và exception vector trở vào đây.
HEX0 ... HEX5 PIO	Slave output	Mỗi PIO xuất 7 hoặc 8 bit ra một LED bảy đoạn. Sáu PIO này được export ra top-level và gán chân đến HEX0 ... HEX5 của kit.
JTAG UART	Slave	Dùng <code>printf</code> để kiểm tra chương trình đã chạy trên Nios II.

Bảng 11 : Sơ đồ khối phần cứng cho Prj1

3.2.2 Mã hiển thị và thứ tự các HEX

Phần mềm buổi 3 có hai ý chính. Thứ nhất là tạo mảng `hex_code[10]` để đổi chữ số thập phân sang mã LED bảy đoạn active-low. Thứ hai là ghi từng chữ số ra đúng vị trí HEX bằng `IOWR(BASE, 0, data)`. Theo slide ví dụ 12 giờ 38 phút 12 giây, thứ tự hiển thị là:

- `HEX0` : hàng đơn vị giây.
- `HEX1` : hàng chục giây.
- `HEX2` : hàng đơn vị phút.
- `HEX3` : hàng chục phút.
- `HEX4` : hàng đơn vị giờ.
- `HEX5` : hàng chục giờ.

```

1  #include "io.h"
2  #include "system.h"
3  #include <unistd.h>
4
5  static const unsigned char hex_code[10] = {
6      0xC0, // 0

```

```

7    0xF9, // 1
8    0xA4, // 2
9    0xB0, // 3
10   0x99, // 4
11   0x92, // 5
12   0x82, // 6
13   0xF8, // 7
14   0x80, // 8
15   0x90 // 9
16 };
17
18 static void display_time(unsigned int hh, unsigned int mm, unsigned
int ss) {
19     IOWR(HEX0_BASE, 0, hex_code[ss % 10]);
20     IOWR(HEX1_BASE, 0, hex_code[ss / 10]);
21     IOWR(HEX2_BASE, 0, hex_code[mm % 10]);
22     IOWR(HEX3_BASE, 0, hex_code[mm / 10]);
23     IOWR(HEX4_BASE, 0, hex_code[hh % 10]);
24     IOWR(HEX5_BASE, 0, hex_code[hh / 10]);
25 }
26
27 int main(void) {
28     unsigned int hh = 0, mm = 0, ss = 0;
29
30     while (1) {
31         display_time(hh, mm, ss);
32         usleep(1000000);
33         if (++ss == 60) { ss = 0; if (++mm == 60) { mm = 0; if (++hh ==
24) hh = 0; } }
34     }
35 }

```


Ghi nhớ. Bản chất Prj1 là CPU tự làm mọi việc: đổi số sang mã bảy đoạn, ghi từng PIO và tạo mốc một giây bằng `usleep` hoặc delay phần mềm. Vì CPU bị chiếm trong lúc đợi nên đây vẫn là cách polling/busy-wait; Prj3 mới chuyển sang Timer để tạo mốc thời gian bằng phần cứng.

3.2.3 Chi tiết cần nắm thêm cho Prj1

Trong Prj1, phần cứng đơn giản nhưng phần mềm phải tự làm khá nhiều việc: tách giá trị giờ/phút/giây thành từng chữ số, đổi chữ số sang mã active-low và ghi đúng thứ tự `HEX0` đến `HEX5`. Nếu giảng viên hỏi sâu, cần nhấn mạnh rằng mỗi PIO chỉ nhận mã đoạn thô; PIO không biết giá trị đó là số nào.

Chi tiết	Cần nói được	Lỗi dễ gặp
Mã active-low	Bit 0 thường nghĩa là đoạn LED sáng; vì vậy mã số 0 là <code>0xC0</code> , không phải <code>0x00</code> .	Dùng mã active-high làm LED hiển thị sai hoặc đảo ngược.
Thứ tự digit	<code>HEX0</code> , <code>HEX1</code> dành cho giây; <code>HEX2</code> , <code>HEX3</code> dành cho phút; <code>HEX4</code> , <code>HEX5</code> dành cho giờ.	Đảo hàng chục/hàng đơn vị khiến ví dụ 12:38:12 hiển thị sai vị trí.
Base address	Mỗi PIO có một macro <code>_BASE</code> riêng trong <code>system.h</code> .	Copy nhầm <code>HEX0_BASE</code> cho cả sáu lần ghi.
<code>usleep</code>	Tạo delay một giây ở phía phần mềm.	Không có timer hệ thống hoặc cấu hình BSP sai làm delay không đúng như mong muốn.

Bảng 12 : Các điểm thường bị hỏi sâu trong Prj1

Ví dụ tách một giá trị thời gian thành hàng chục và hàng đơn vị:

```
1 unsigned int sec_unit = ss % 10;
2 unsigned int sec_tens = ss / 10;
```



Ưu và nhược điểm của Prj1

Ưu điểm là rất dễ nhìn thấy quan hệ CPU - Avalon Bus - PIO: mỗi lệnh `IOWR` tạo một giao tác ghi đến một Slave. Nhược điểm là phải dùng sáu PIO riêng, phần mềm phải tự mã hóa LED bảy đoạn và CPU vẫn bị chiếm bởi delay.

3.3. Prj2 — Custom IP HEX điều khiển sáu LED bảy đoạn

3.3.1 Mục tiêu phần cứng

Theo file `Custome_IP_hex.pdf`, Prj2 tập trung vào việc tự viết một IP tên `HEX` để điều khiển trực tiếp sáu LED bảy đoạn. IP này được đóng gói thành một Avalon-MM Slave write-only có các tín hiệu chính:

- `iClk`, `iRst_n`: clock và reset active-low.
- `iChipSelect`: Bus chọn đúng Slave `hex_0`.
- `iAddress[2:0]`: chọn một trong sáu digit, thường dùng địa chỉ 0 đến 5.
- `iWrite`: yêu cầu ghi active-high theo khai báo trong Component Editor.
- `iWriteData[31:0]`: dữ liệu CPU ghi xuống; IP thường dùng vài bit thấp để lấy chữ số 0 đến 9.
- `oHex0` đến `oHex5`: sáu ngõ ra conduit, mỗi ngõ 7 bit ra LED bảy đoạn.

Điểm khác Prj1 là phần mềm không ghi mã đoạn thô cho từng PIO riêng nữa. Phần mềm chỉ ghi chữ số vào cùng một base address `HEX_0_BASE`, còn custom IP tự giải mã chữ số thành mã LED bảy đoạn và xuất ra `oHex0` ... `oHex5`.

Thành phần	Loại interface	Mục đích
avalon_slave_0	Avalon-MM Slave	Nhận <code>chipselct</code> , <code>address</code> , <code>write</code> , <code>writedata</code> từ Bus. Không cần <code>read</code> / <code>readdata</code> nếu bài chỉ yêu cầu ghi hiển thị.
clock_sink	Clock input	Nối clock hệ thống <code>clk_0</code> cho logic đồng bộ trong IP.
reset_sink	Reset input	Nối reset active-low <code>iRst_n</code> để đưa sáu HEX về trạng thái xác định.
conduit_end	External conduit	Export sáu output <code>hex0</code> ... <code>hex5</code> ra top-level rồi gán chân đến LED bảy đoạn trên kit.

Bảng 13 : Các interface của custom IP HEX trong Component Editor

3.3.2 Mã HDL khái quát của IP

Phần HDL cần hai khối rõ ràng. Khối thứ nhất là hàm đổi chữ số 0 đến 9 sang mã bảy đoạn active-low. Khối thứ hai là logic ghi đồng bộ: khi `iChipSelect && iWrite`, IP dùng `iAddress` để chọn `oHex0` ... `oHex5` và dùng `iWriteData[3:0]` làm chữ số cần hiển thị.

```

1  module HEX(
2      input      iClk,
3      input      iRst_n,
4      input      iChipSelect,
5      input [2:0] iAddress,
6      input      iWrite,
7      input [31:0] iWriteData,
8      output reg [6:0] oHex0,
9      output reg [6:0] oHex1,
10     output reg [6:0] oHex2,
11     output reg [6:0] oHex3,
12     output reg [6:0] oHex4,

```

```

13     output reg [6:0] oHex5
14 );
15
16 function [6:0] seg7;
17     input [3:0] digit;
18     begin
19         case (digit)
20             4'd0: seg7 = 7'b1000000;
21             4'd1: seg7 = 7'b1111001;
22             4'd2: seg7 = 7'b0100100;
23             4'd3: seg7 = 7'b0110000;
24             4'd4: seg7 = 7'b0011001;
25             4'd5: seg7 = 7'b0010010;
26             4'd6: seg7 = 7'b0000010;
27             4'd7: seg7 = 7'b1111000;
28             4'd8: seg7 = 7'b0000000;
29             4'd9: seg7 = 7'b0010000;
30             default: seg7 = 7'b1111111;
31         endcase
32     end
33 endfunction
34
35 always @(posedge iClk or negedge iRst_n) begin
36     if (!iRst_n) begin
37         oHex0 <= 7'b1111111;
38         oHex1 <= 7'b1111111;
39         oHex2 <= 7'b1111111;
40         oHex3 <= 7'b1111111;
41         oHex4 <= 7'b1111111;
42         oHex5 <= 7'b1111111;
43     end else if (iChipSelect && iWrite) begin
44         case (iAddress)

```

```

45      3'd0: oHex0 <= seg7(iWriteData[3:0]);
46      3'd1: oHex1 <= seg7(iWriteData[3:0]);
47      3'd2: oHex2 <= seg7(iWriteData[3:0]);
48      3'd3: oHex3 <= seg7(iWriteData[3:0]);
49      3'd4: oHex4 <= seg7(iWriteData[3:0]);
50      3'd5: oHex5 <= seg7(iWriteData[3:0]);
51      default: ;
52  endcase
53  end
54 end
55
56 endmodule

```

3.3.3 Phần mềm

Trong phần mềm, chỉ cần include `io.h` và `system.h`, sau đó gọi `IOWR(HEX_0_BASE, index, digit)`. Tham số `index` chọn digit cần ghi, còn `digit` là giá trị 0 đến 9. Đây là đúng tinh thần slide buổi 4: cùng một custom IP, một base address, sáu offset.

```

1  #include "io.h"
2  #include "system.h"
3  #include <stdio.h>
4
5  int main(void) {
6      printf("Hien thi len Hex hoan tat");
7
8      while (1) {
9          IOWR(HEX_0_BASE, 0, 0);
10         IOWR(HEX_0_BASE, 1, 1);
11         IOWR(HEX_0_BASE, 2, 2);
12         IOWR(HEX_0_BASE, 3, 3);
13         IOWR(HEX_0_BASE, 4, 4);
14         IOWR(HEX_0_BASE, 5, 5);

```

```
15    }
16 }
```

Nếu muốn dùng custom IP này để hiển thị đồng hồ như Prj1, phần mềm chỉ thay sáu giá trị 0..5 bằng chữ số của giờ/phút/giây:

```
1  static void display_time_custom_ip(unsigned int hh, unsigned int
mm, unsigned int ss) {
2      IOWR(HEX_0_BASE, 0, ss % 10);
3      IOWR(HEX_0_BASE, 1, ss / 10);
4      IOWR(HEX_0_BASE, 2, mm % 10);
5      IOWR(HEX_0_BASE, 3, mm / 10);
6      IOWR(HEX_0_BASE, 4, hh % 10);
7      IOWR(HEX_0_BASE, 5, hh / 10);
8  }
```

Tại sao tự viết IP

Mục tiêu của Prj2 là chứng minh sinh viên hiểu cách **đóng gói một IP Avalon-MM Slave**: viết HDL, khai báo signal type trong Component Editor, export conduit ra top-level, kết nối CPU Master đến Slave `hex_0`, và truy cập IP từ C bằng macro `_BASE` trong `system.h`.

3.3.4 Đóng gói IP tự viết trong Component Editor

Khi tự viết IP, phần HDL mới chỉ là lõi logic. Để Platform Designer hiểu đây là một Avalon-MM Slave, sinh viên phải dùng Component Editor khai báo interface, ánh xạ tên port và đặt thông số dữ liệu. Slide buổi 4 thể hiện rõ bảng Signals: `iChipSelect` là `chipselect`, `iAddress` là `address` rộng 3 bit, `iWrite` là `write`, `iWriteData` là `writedata` rộng 32 bit; `oHex0` ... `oHex5` là các output conduit rộng 7 bit.

Mục trong Component Editor	Ý nghĩa
Files	Thêm file Verilog chứa module <code>HEX</code> , kiểm tra tên module đúng với tên entity top.
Signals	Ánh xạ <code>iChipSelect</code> , <code>iAddress</code> , <code>iWrite</code> , <code>iWriteData</code> , <code>iClk</code> , <code>iRst_n</code> , <code>oHex0</code> ... <code>oHex5</code> đúng signal type.
Interfaces	Gom bốn tín hiệu bus thành <code>avalon_slave_0</code> , gom <code>iClk</code> thành <code>clock_sink</code> , gom <code>iRst_n</code> thành <code>reset_sink</code> , gom sáu output thành <code>conduit_end</code> .
Address width	Dùng 3 bit vì cần chọn tối đa sáu vị trí HEX, tương ứng index 0 đến 5 trong <code>IOWR</code> .
Data width	Dùng <code>writedata</code> 32 bit theo bus, nhưng IP chỉ cần vài bit thấp để nhận chữ số.
Export conduit	Sáu output <code>hex0</code> ... <code>hex5</code> phải export ra hệ thống và nối ra chân vật lý của kit.

Bảng 14 : Các mục cần kiểm tra khi đóng gói IP tự viết

3.3.5 Simulation và lỗi thường gặp trong buổi 4

Sau khi Generate HDL trong Platform Designer, slide yêu cầu chọn thêm **Generate Testbench System** để tạo môi trường mô phỏng. Khi mở ModelSim/Quarta, sinh viên thêm các tín hiệu của IP vào waveform, chạy `run 500 ms` và quan sát `iAddress`, `iWrite`, `iWriteData`, `oHex0` ... `oHex5` thay đổi đúng theo lệnh `IOWR`.

Tình huống	Cách hiểu/cách xử lý
Không thấy <code>HEX_0_BASE</code>	Chưa Generate hệ thống hoặc chưa sinh lại BSP. Mở <code>system.h</code> để kiểm tra tên macro đúng với instance <code>hex_0</code> .
Ghi <code>IOWR</code> nhưng HEX không đổi	Kiểm tra <code>iChipSelect</code> , <code>iWrite</code> , <code>iAddress</code> trong waveform; kiểm tra conduit <code>hex</code> đã export và nối pin.
ModelSim báo không tìm thấy path	Kiểm tra lại run configuration của Nios II/ModelSim và đường dẫn thư mục simulation/submodules theo slide sửa lỗi.
Chương trình C quá lớn	Trong BSP Editor có thể bật <code>enable_small_c_library</code> , <code>enable_reduced_device_drivers</code> , <code>enable_sim_optimize</code> như slide gợi ý.

Bảng 15 : Các lỗi thực hành cần nhớ cho Prj2

Ghi nhớ. Vì IP buổi 4 là write-only nên việc không có `readdata` không phải lỗi. Avalon cho phép một Slave chỉ khai báo các tín hiệu cần dùng. Điều quan trọng là `iWrite` trong slide là active-high, khác với các ví dụ manual cũ thường dùng tên `write_n` active-low.

3.3.6 So sánh Prj1 và Prj2

Tiêu chí	Prj1	Prj2
Khởi hiển thị	Sáu PIO riêng HEX0 ... HEX5 ; phần mềm ghi mã đoạn trực tiếp.	Một custom IP hex_0 ; phần mềm ghi chữ số, HDL tự giải mã ra mã đoạn.
Base address	Mỗi HEX có một _BASE riêng.	Một HEX_0_BASE , địa chỉ nội bộ 0 đến 5 chọn digit.
Tín hiệu Avalon cần nhớ	PIO là Slave có sẵn, sinh viên chủ yếu thao tác từ C.	Phải tự khai báo chipselect , address , write , writedata và conduit output.
Điểm thi chính	Biết dùng IP sẵn có và hiểu mỗi IOWR là một giao tác ghi.	Biết tự viết, đóng gói, tích hợp, mô phỏng và gọi IP từ phần mềm.

Bảng 16 : So sánh Prj1 và Prj2

3.4. Prj3 — Đồng hồ dùng Timer cho phần delay một giây

3.4.1 Mục tiêu phần cứng

Prj3 thay vòng lặp CPU bằng IP Timer của Altera. Timer được cấu hình ngắt mỗi 1 giây (đặt period = clock_freq để Timer đếm xuống và roll-over đúng 1 Hz). Khi Timer hết khoảng đếm, nó phát IRQ về CPU; ISR cập nhật biến giây/phút/giờ và xuất ra HEX. CPU không cần busy-wait nữa, có thể vào trạng thái idle hoặc làm việc khác giữa hai ngắt.

IP	Vai trò	Mục đích
Nios II	Master	Chạy chương trình C, xử lý ngắt từ Timer.
On-chip memory	Slave	Code, stack, exception vector.
Interval Timer	Slave + IRQ	Sinh ngắt mỗi 1 giây để cập nhật giờ/phút/giây.
PIO_HEX	Slave (output)	Hiển thị thời gian.
PIO_SW	Slave (input)	Đọc thiết lập ban đầu.

Bảng 17 : Sơ đồ khối phần cứng cho Prj3

3.4.2 Phần mềm dùng ISR

```
1  #include "system.h"
2  #include "altera_avalon_timer_regs.h"
3  #include "sys/alt_irq.h"
4
5  static volatile unsigned int hh, mm, ss;
6
7  static void timer_isr(void *context) {
8      // Xóa cờ TO của Timer để cho phép ngắt tiếp theo.
9      IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_BASE, 0);
10     if (++ss == 60) { ss = 0; if (++mm == 60) { mm = 0; if (++hh == 24)
        hh = 0; } }
11 }
12
13 int main(void) {
14     // Cấu hình period 1 Hz, bật continuous + interrupt + start.
15     IOWR_ALTERA_AVALON_TIMER_PERIODL(TIMER_BASE, (TIMER_FREQ - 1) &
        0xFFFF);
16     IOWR_ALTERA_AVALON_TIMER_PERIODH(TIMER_BASE, ((TIMER_FREQ - 1) >>
        16) & 0xFFFF);
17     IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_BASE, 0x7);
18     alt_ic_isr_register(TIMER_IRQ_INTERRUPT_CONTROLLER_ID, TIMER_IRQ,
        timer_isr, 0, 0);
19
20     while (1) {
21         unsigned int hex = (hh << 12) | (mm << 6) | ss;
22         IOWR_ALTERA_AVALON_PIO_DATA(PIO_HEX_BASE, hex);
23     }
24 }
```

Ghi nhớ. Trong ISR phải xóa cờ TO (timeout) bằng cách ghi 0 vào thanh ghi STATUS của Timer. Quên bước này khiến ngắt sẽ liên tục được tái phát, treo CPU.

3.4.3 Các thanh ghi Timer cần nhớ

IP Interval Timer của Altera là một Slave có nhiều thanh ghi điều khiển. Khi vấn đáp, không cần thuộc mọi bit, nhưng nên nắm được bốn nhóm thanh ghi chính:

Thanh ghi	Hướng	Ý nghĩa
STATUS	Read/Write	Chứa cờ timeout <code>T0</code> . ISR phải xóa cờ này sau khi xử lý ngắt.
CONTROL	Read/Write	Bật interrupt (<code>IT0</code>), chế độ continuous (<code>CONT</code>), start/stop Timer. Giá trị thường dùng để chạy liên tục có ngắt là <code>0x7</code> .
PERIODL	Write	16 bit thấp của chu kỳ đếm. Với clock 50 MHz, period 1 giây thường là 50,000,000 - 1.
PERIODH	Write	16 bit cao của chu kỳ đếm. Phải ghi đủ cả low và high nếu period lớn hơn 16 bit.
SNAPL / SNAPH	Read	Đọc giá trị đếm hiện tại khi cần đo thời gian hoặc debug, không bắt buộc trong bài đồng hồ đơn giản.

Bảng 18 : Các thanh ghi Timer thường dùng trong Prj3

3.4.4 Trình tự cấu hình hình ngắt Timer

Thứ tự cấu hình nên làm rõ vì nếu đảo lung tung, chương trình có thể nhận ngắt trước khi biến và ISR sẵn sàng.

1. Khai báo biến thời gian dùng trong ISR là `volatile`.
2. Đọc Switch để lấy giờ/phút/giây ban đầu, chuẩn hóa về khoảng hợp lệ.
3. Ghi `PERIODL` và `PERIODH` để đặt chu kỳ 1 giây.
4. Đăng ký ISR bằng `alt_ic_isr_register(...)`.
5. Xóa cờ timeout cũ trong `STATUS`.
6. Ghi `CONTROL` để bật `IT0`, `CONT` và `START`.
7. Trong vòng `while(1)`, main chỉ hiển thị hoặc xử lý việc phụ; phần tăng giây nằm trong ISR.

Vì sao biến trong ISR phải volatile

Biến `hh`, `mm`, `ss` được thay đổi trong ISR nhưng lại được đọc ở hàm `main`. Nếu không khai báo `volatile`, compiler có thể giữ giá trị cũ trong thanh ghi CPU và main không nhìn thấy cập nhật mới. Đây là lỗi phần mềm nhưng hậu quả quan sát giống như Timer không chạy.

3.4.5 So sánh Prj1 và Prj3 về thời gian

Tiêu chí	Delay CPU trong Prj1	Timer IRQ trong Prj3
Độ chính xác	Phụ thuộc vòng lặp, compiler, tần số CPU.	Phụ thuộc Timer phần cứng, ổn định hơn nhiều.
Tải CPU	CPU bận trong lúc delay.	CPU rảnh giữa hai ngắt.
Khả năng mở rộng	Khó xử lý sự kiện khác đúng lúc.	Dễ thêm nút nhấn, UART hoặc xử lý nền.
Độ phức tạp	Dễ viết, ít cấu hình.	Cần biết IRQ, ISR, thanh ghi Timer.

Bảng 19 : Ưu nhược điểm của hai cách tạo mốc 1 giây

3.5. Prj4 — Hệ thống có dùng DMAC

3.5.1 Mục tiêu phần cứng

Prj4 không nhất thiết là một đồng hồ. Mục tiêu chính là minh họa khái niệm **DMA**: chuyển một khối dữ liệu từ vùng nhớ nguồn sang vùng nhớ đích (hoặc tới một thiết bị I/O) mà không cần CPU đọc/ghi từng byte. DMAC là một IP đặc biệt vì nó vừa là **Slave** để CPU đọc/ghi thanh ghi cấu hình, vừa là **Master** để tự sinh địa chỉ truy cập nguồn và đích.

Hệ thống điển hình gồm: CPU, on-chip memory chứa code, một vùng nhớ nguồn (có thể là on-chip memory thứ hai chứa dữ liệu mẫu), một vùng nhớ đích (on-chip memory hoặc PIO output cho LED/HEX), và DMAC. Khi CPU cấu hình DMAC, DMAC sẽ tự đọc nguồn và ghi đích cho đến khi xong khối dữ liệu.

IP	Vai trò	Mục đích
Nios II	Master	Cấu hình DMAC, chờ tín hiệu hoàn tất.
DMAC	Master + Slave	Slave để CPU ghi cấu hình; Master để tự đọc nguồn và ghi đích.
On-chip memory SRC	Slave	Vùng nguồn chứa dữ liệu mẫu cần chuyển.
On-chip memory DST hoặc PIO	Slave	Vùng đích nhận dữ liệu.
JTAG UART	Slave	<code>printf</code> để xác nhận DMA hoàn tất.

Bảng 20 : Sơ đồ khối phần cứng cho Prj4

3.5.2 Trình tự chạy DMA điển hình

1. CPU ghi địa chỉ nguồn vào thanh ghi `DMA_SRC` của DMAC.
2. CPU ghi địa chỉ đích vào thanh ghi `DMA_DST`.
3. CPU ghi số byte cần chuyển vào thanh ghi `DMA_LEN`.
4. CPU ghi vào thanh ghi `DMA_CONTROL` để khởi động (bật RUN, chọn chiều, chế độ word/byte, có dừng ngắt khi xong hay không).
5. DMAC tự thực hiện vòng đọc nguồn → ghi đích, cho đến khi đủ số byte. Trong khi đó, DMAC đóng vai trò Master trên Bus.
6. Khi xong, DMAC đặt cờ DONE hoặc phát IRQ. CPU đọc kết quả ở vùng đích.

```

1 // Khái quát đoạn cấu hình DMAC bằng macro của HAL Altera
2 #include "sys/alt_dma.h"
3 static volatile int dma_done = 0;
4 static void dma_done_callback(void *context, void *buffer) { dma_done
  = 1; }
5
6 void run_dma(void *src, void *dst, int len) {
7     alt_dma_txchan tx = alt_dma_txchan_open("/dev/dma_0");
8     alt_dma_rxchan rx = alt_dma_rxchan_open("/dev/dma_0");
9     alt_dma_rxchan_prepare(rx, dst, len, dma_done_callback, NULL);

```

```

10 alt_dma_txchan_send(tx, src, len, NULL, NULL);
11 while (!dma_done) { }
12 }

```

Vì sao cần DMAC

Khi cần chuyển một khối lớn dữ liệu, ví dụ buffer ảnh hoặc sample âm thanh, nếu để CPU làm `for` từng byte thì CPU bị chiếm hết cho phần truyền dữ liệu. DMAC giải phóng CPU bằng cách tự đảm nhiệm vòng đọc/ghi. Điểm thi chính là **DMAC vừa là Master vừa là Slave**, và quy trình **cấu hình – khởi động – chờ hoàn tất**.

3.5.3 DMAC vừa là Slave vừa là Master như thế nào?

Một cách giải thích rất dễ hiểu là chia quá trình DMA thành hai pha. Ở pha cấu hình, CPU là Master và DMAC là Slave: CPU ghi địa chỉ nguồn, địa chỉ đích, độ dài và bit start vào các thanh ghi DMAC. Ở pha truyền dữ liệu, DMAC trở thành Master: nó tự phát địa chỉ đọc đến Slave nguồn và địa chỉ ghi đến Slave đích. CPU không còn đọc/ghi từng byte nữa.

Pha	Vai trò của CPU	Vai trò của DMAC
Cấu hình	Master, ghi thanh ghi điều khiển của DMAC.	Slave, nhận thông số từ CPU.
Truyền dữ liệu	Không trực tiếp truyền từng word; có thể polling hoặc chờ IRQ.	Master, tự đọc nguồn và ghi đích qua Avalon Bus.
Hoàn tất	Đọc cờ DONE hoặc chạy ISR khi có IRQ.	Cập nhật trạng thái, phát IRQ nếu được bật.

Bảng 21 : Hai pha hoạt động của DMAC

3.5.4 Các điểm cần chú ý khi dùng DMA

Vấn đề	Giải thích
Địa chỉ nguồn/ đích	Phải là địa chỉ mà DMAC truy cập được trên Avalon Bus. Không phải mọi địa chỉ con trỏ C đều phù hợp nếu vùng đó không nằm trong memory map của hệ thống.
Độ dài truyền	Cần thống nhất đơn vị byte hay word. Một số API nhận số byte, trong khi người học dễ nhầm thành số phần tử.
Alignment	Truyền word thường yêu cầu địa chỉ căn theo 2 hoặc 4 byte. Sai alignment có thể làm truyền chậm hoặc sai dữ liệu.
Cache	Nếu CPU có data cache, dữ liệu CPU thấy và dữ liệu DMAC thấy có thể không đồng nhất. Cần flush/invalidate cache hoặc dùng vùng nhớ không cache khi bài có yêu cầu.
Polling và IRQ	Polling đơn giản nhưng CPU vẫn phải chờ; IRQ tốt hơn khi muốn CPU làm việc khác trong lúc DMA chạy.
Arbitration	DMAC và CPU có thể cùng tranh truy cập memory. Avalon Bus phải trọng tài để quyết định Master nào được phục vụ.

Bảng 22 : Các rủi ro thực tế khi làm Prj4

3.5.5 Cách kiểm chứng DMA đã chạy đúng

Khi debug Prj4, không nên truyền ngay một buffer lớn. Nên bắt đầu bằng một mảng nhỏ có mẫu dễ nhận ra, ví dụ `0x11, 0x22, 0x33, 0x44`. Sau khi DMA xong, CPU in vùng đích ra JTAG UART hoặc xuất một vài byte ra LED/HEX. Nếu vùng đích đúng, mới tăng dần độ dài truyền.

```
1 unsigned char src[4] = {0x11, 0x22, 0x33, 0x44};
2 unsigned char dst[4] = {0x00, 0x00, 0x00, 0x00};
3
4 run_dma(src, dst, 4);
5 printf("%02x %02x %02x %02x\n", dst[0], dst[1], dst[2], dst[3]);
```

Ghi nhớ. Nếu dùng DMA để đưa dữ liệu ra PIO/HEX, cần nhớ PIO là thiết bị I/O chứ không phải bộ nhớ liên tục. Vì vậy cách cấu hình tăng địa chỉ đích hay giữ cố định địa chỉ đích phải phù hợp với loại thiết bị nhận.

3.6. So sánh nhanh bốn project

Prj	Cách đếm 1 giây	Khối điều khiển hiển thị	Điểm trọng tâm
Prj1	<code>usleep</code> hoặc delay phần mềm trong CPU.	Sáu IP PIO riêng cho <code>HEX0</code> ... <code>HEX5</code> ; phần mềm ghi mã seven-segment active-low.	Làm quen với pipeline Platform Designer → BSP → C → HEX.
Prj2	Có thể dùng lại thuật toán phần mềm của Prj1.	Một custom IP <code>hex_0</code> , địa chỉ 0..5 chọn digit, <code>writedata</code> chứa chữ số.	Kỹ thuật viết, đóng gói, tích hợp và mô phỏng một Avalon-MM Slave write-only.
Prj3	Timer phát ngắt 1 Hz; CPU cập nhật biến trong ISR.	PIO HEX vẫn từ thư viện.	Cơ chế ngắt (IRQ, ISR), giải phóng CPU khỏi busy-wait.
Prj4	Không tập trung đếm giây; tập trung truyền khối dữ liệu.	Vùng đích là on-chip memory hoặc PIO output.	DMAC là Master + Slave; quy trình cấu hình và chờ hoàn tất.

Bảng 23 : So sánh nhanh đặc điểm bốn project trong đề cương

3.7. Mẫu trả lời vấn đáp cho từng project

Prj1

Prj1 dùng CPU Nios II làm Master, sáu PIO `HEX0` ... `HEX5` làm Slave. CPU tự đổi chữ số sang mã seven-segment active-low rồi ghi từng PIO bằng `IOWR(HEXx_BASE, 0, data)`. Trọng tâm là biết dùng IP có sẵn, biết thứ tự digit giờ/phút/giây và hiểu mỗi lần `IOWR` là một giao tác ghi Avalon-MM.

Prj2

Prj2 thay sáu PIO bằng một custom IP `HEX` đóng vai trò Avalon-MM Slave write-only. CPU ghi `IOWR(HEX_0_BASE, index, digit)`, trong đó `index` chọn `oHex0` ... `oHex5`, còn `digit` là số 0 đến 9. Trọng tâm là biết viết HDL giải mã `iAddress`, dùng `iChipSelect`, `iWrite`, `iWriteData`, export sáu conduit output, rồi đóng gói IP bằng Component Editor.

Prj3

Prj3 dùng Interval Timer để tạo mốc 1 giây bằng ngắt, thay cho delay CPU. Timer là Slave có IRQ; CPU cấu hình period, đăng ký ISR, bật interrupt và trong ISR xóa cờ timeout rồi cập nhật thời gian. Trọng tâm là hiểu cơ chế IRQ/ISR và lý do Timer chính xác hơn busy-wait.

Prj4

Prj4 dùng DMAC để truyền khối dữ liệu. CPU cấu hình DMAC thông qua các thanh ghi nên lúc này DMAC là Slave; khi bắt đầu truyền, DMAC tự sinh giao tác đọc/ghi trên Bus nên nó là Master. Trọng tâm là quy trình cấu hình nguồn, đích, độ dài, start, chờ DONE/IRQ và hiểu arbitration khi có nhiều Master.

3.8. Câu hỏi ôn tập gợi ý

Ghi nhớ.

1. Vẽ sơ đồ khối phần cứng cho từng project. Tại mỗi project, chỉ rõ Master, Slave và lý do chọn IP đó.
2. Trong Prj1, vì sao phải dùng bảng `hex_code[10]` thay vì ghi thẳng số 0 đến 9 ra PIO?
3. Trong Prj2, vì sao custom IP chỉ cần `write` / `writedata` mà không nhất thiết phải có `read` / `readdata`?
4. Trong Prj3, ISR của Timer cần làm gì để đảm bảo ngắt tiếp theo vẫn hoạt động bình thường?

5. Trong Prj4, DMAC vừa là Master vừa là Slave nghĩa là gì? Nêu các thanh ghi cấu hình tối thiểu cho một lượt DMA.
6. So sánh ưu nhược điểm của Prj1 và Prj3 trong việc đếm giây.
7. Nếu Prj2 ghi `IOWR(HEX_0_BASE, 0, 0)` nhưng `HEX0` không đổi, nên kiểm tra `iChipSelect`, `iWrite`, `iAddress`, `iWriteData`, conduit export hay pin assignment trước?
8. Nếu Prj3 Timer có ngắt nhưng `HEX` không đổi, lỗi có thể nằm ở ISR, biến `volatile`, hay PIO output? Hãy nêu cách khoanh vùng.

CHƯƠNG 4

PHÂN TÍCH HOẠT ĐỘNG CỦA MASTER, BUS, SLAVE TRÊN HÌNH 1 – HÌNH 11

4.1. Cách đọc giản đồ thời gian Avalon

Toàn bộ Hình 1 – Hình 11 trong đề cương được đọc theo manual Avalon Bus cũ của Altera. Nửa trên của hình thường là sơ đồ khối Peripheral → Master Port → Avalon Bus Module → Slave Port → Peripheral; nửa dưới là giản đồ thời gian các tín hiệu chính như `clk`, `address`, `byteenable_n`, `read_n` hoặc `write_n`, `chipsselect`, `readdata` hoặc `writedata`, và có thể có `waitrequest`. Các cột thời gian được đánh nhãn A, B, C, D, ... để tiện chỉ ra ranh giới sự kiện.

Khi đọc giản đồ, hãy giữ ba câu hỏi:

1. **Tín hiệu này do ai phát ra?** — Master hay Slave hay Bus tạo ra. Ví dụ `address`, `read_n`, `write_n`, `writedata` do Master phát; `chipsselect` do Bus phát; `readdata`, `waitrequest` do Slave phát.
2. **Tín hiệu này có hiệu lực ở chu kỳ nào?** — quan sát cạnh xuống/cạnh lên của `clk`, đối chiếu với cột thời gian.
3. **Có chu kỳ chờ không?** — chu kỳ chờ thể hiện qua việc `address`, `chipsselect`, `read_n`/`write_n` được giữ không đổi qua nhiều chu kỳ. Nếu có `waitrequest`, đó là chờ “động”; nếu không có thì là chờ “tĩnh” theo cấu hình Slave.

Quy ước cột thời gian trong tài liệu này

Vì các Hình 1 – Hình 11 nằm trong đề cương gốc, tài liệu chỉ mô tả ý nghĩa từng cột A, B, C, ... chứ không vẽ lại giản đồ. Khi đọc manual, cần chú ý Hình 8 đến Hình 11 là góc nhìn **Master Port**, còn Hình 1 đến Hình 7 là góc nhìn **Slave Port**.

4.2. Cách nhận dạng nhanh một giản đồ Avalon chưa biết

Khi gặp một giản đồ mới, có thể nhận dạng theo bốn bước. Cách này giúp tránh trả lời lan man khi giảng viên hỏi bất kỳ Hình 1 - Hình 11 hoặc vẽ thêm một biến thể.

Bước	Cần quan sát	Kết luận rút ra
1	Tín hiệu điều khiển đang active là <code>read_n</code> hay <code>write_n</code> ?	<code>read_n = 0</code> là giao tác đọc; <code>write_n = 0</code> là giao tác ghi.
2	Đường dữ liệu xuất hiện là <code>readdata</code> hay <code>writedata</code> ?	<code>readdata</code> đi từ Slave về Master; <code>writedata</code> đi từ Master sang Slave.
3	Có tín hiệu <code>waitrequest</code> không? Nếu có, nó ở mức 1 bao lâu?	Có chu kỳ chờ động; Master phải giữ nguyên tín hiệu cho đến khi <code>waitrequest = 0</code> .
4	Nếu không có <code>waitrequest</code> , địa chỉ/control bị giữ qua bao nhiêu chu kỳ?	Đó là số fixed wait state do cấu hình Slave/Bus chèn vào.
5	Có bao nhiêu lần địa chỉ đổi sang giá trị mới?	Mỗi lần địa chỉ mới thường là một giao tác mới; nếu đổi ngay sau khi xong giao tác trước thì là back-to-back.

Bảng 24 : Quy trình nhận dạng một giản đồ thời gian Avalon

Mẹo đếm chu kỳ chờ

Với fixed wait state, hãy đếm số chu kỳ giữa lúc `address` và `read_n/write_n` có hiệu lực đến trước lúc dữ liệu được lấy mẫu hoặc được chốt. Với `waitrequest`, chỉ cần đếm số chu kỳ `waitrequest = 1`; trong toàn bộ khoảng đó Master chưa được kết thúc giao tác.

4.3. Bảng ai phát tín hiệu trong giản đồ

Một giản đồ Avalon có nhiều đường tín hiệu nên rất dễ nhầm nguồn phát. Bảng sau giúp trả lời nhanh câu hỏi “tín hiệu này do ai tạo ra?”.

Tín hiệu	Nguồn phát chính	Ghi chú khi đọc giản đồ
address	Master	Bus có thể chuyển đổi hoặc cắt bớt thành offset trước khi đưa tới Slave.
byteenable_n	Master	Đi kèm địa chỉ để chỉ byte hợp lệ trong word. Active-low.
read_n	Master	Active-low; khi bằng 0 nghĩa là đang yêu cầu đọc.
write_n	Master	Active-low; khi bằng 0 nghĩa là đang yêu cầu ghi.
writedata	Master	Phải ổn định trong toàn bộ giao tác ghi, nhất là khi <code>waitrequest = 1</code> .
chipsselect	Bus/ interconnect	Được sinh ra sau khi Bus giải mã địa chỉ và chọn đúng Slave.
readdata	Slave	Chỉ có ý nghĩa trong giao tác đọc và phải hợp lệ trước lúc Master lấy mẫu.
waitrequest	Slave hoặc interconnect	Kéo lên 1 để yêu cầu Master giữ giao tác.

Bảng 25 : Nguồn phát của các tín hiệu trong giản đồ Avalon-MM

4.4. Hình 1 — Slave Read cơ bản (zero wait state)

4.4.1 Bối cảnh

Hình 1 minh họa giao tác đọc đơn giản nhất: Slave đáp ứng ngay trong chu kỳ kế tiếp, không cần chèn thêm chu kỳ chờ. Đây là dạng giao thức quen thuộc với các Slave có độ trễ nội bộ không đáng kể, ví dụ thanh ghi PIO đơn giản. Sơ đồ khối phía trên cho thấy Master Port phát Address, Data, Control vào Avalon Bus Module; Bus phát ra address, byteenable_n, read_n, chipsselect, readdata đối diện với Slave Port.

4.4.2 Phân tích từng cột

Cột A — chu kỳ nghỉ trước giao tác. `address`, `read_n`, `chipsselect` ở mức không hoạt động (`read_n` = 1, `chipsselect` = 0). Bus và Slave không làm gì.

Cột B — Master phát địa chỉ và `byteenable_n`; `read_n` xuống 0; Bus giải mã và đặt `chipsselect` lên 1 cho đúng Slave.

Cột C — Slave nhận diện được giao tác đọc (`chipsselect` = 1, `read_n` = 0). Slave xuất giá trị thanh ghi tương ứng lên `readdata`.

Cột D — Master lấy mẫu `readdata` ở cạnh lên `clk`. Đây cũng là thời điểm dữ liệu được CPU coi là “đã có”.

Cột E — Master kết thúc giao tác; `read_n` về 1, `chipsselect` về 0. Hệ thống quay lại trạng thái nghỉ.

Đặc điểm cần nhớ về Hình 1

Hình 1 là tham chiếu để so sánh với các hình khác. Toàn bộ giao tác chỉ chiếm một chu kỳ “có ích” và một vài chu kỳ thiết lập/kết thúc. Đây là zero wait state cho Slave Read.

4.5. Hình 2 — Slave Read với 1 chu kỳ chờ (fixed wait state = 1)

Hình 2 lặp lại giao thức của Hình 1 nhưng Slave được khai báo có 1 fixed wait state. Trong cột thời gian, ta thấy thêm một chu kỳ giữa “address valid” và “readdata valid” so với Hình 1.

Cột A — chu kỳ nghỉ.

Cột B — Master phát `address`, `byteenable_n`; `read_n` xuống 0. Bus đặt `chipsselect` lên 1.

Cột C — Slave bắt đầu xử lý nhưng chưa kịp xuất `readdata`. Bus **không** phát `waitrequest` — wait state này là “tĩnh”, do Qsys cấu hình sẵn. Từ phía Master, các tín hiệu `address`, `read_n`, `chipsselect` được giữ nguyên.

Cột D — Slave xuất `readdata` hợp lệ.

Cột E — Master lấy mẫu `readdata`.

Cột F — Master kết thúc giao tác, các tín hiệu trở về trạng thái không hoạt động.

Ghi nhớ. Khi cấu hình Slave với fixed wait state, chính Bus chèn thêm chu kỳ và giữ tín hiệu giúp Master. Master không cần biết Slave chậm bao nhiêu chu kỳ; nó chỉ cần “đợi tới chu kỳ đọc dữ liệu” theo quy ước.

4.6. Hình 3 — Slave Read với 2 chu kỳ chờ (fixed wait state = 2)

Hình 3 mở rộng tiếp Hình 2 với 2 chu kỳ chờ tĩnh.

Cột A — chu kỳ nghỉ.

Cột B — Master phát `address`, `byteenable_n`, `read_n` xuống 0; Bus đặt `chipsselect` lên 1.

Cột C, D — hai chu kỳ chờ tĩnh. `address`, `read_n`, `chipsselect` được giữ nguyên. Slave đang xử lý nhưng chưa xuất `readdata`.

Cột E — Slave xuất `readdata` hợp lệ.

Cột F — Master lấy mẫu `readdata`. Sau đó các tín hiệu trở về không hoạt động.

Khi nào dùng nhiều fixed wait state

Slave có thể cần nhiều fixed wait state khi truy cập một nguồn chậm hơn, ví dụ thanh ghi nội bộ chia clock hoặc bộ nhớ đồng bộ có độ trễ đọc cố định. Sự khác biệt giữa Hình 1, Hình 2, Hình 3 chỉ là **số chu kỳ chờ tĩnh**; giao thức và vai trò các tín hiệu giữ nguyên.

4.7. Hình 4 — Slave Read với waitrequest (variable wait state)

4.7.1 Đặc điểm

Hình 4 thay fixed wait state bằng tín hiệu `waitrequest` do Slave phát. Đây là cách quản lý độ trễ động: Slave chủ động báo cho Bus rằng “tôi chưa xong, đừng lấy dữ liệu vội”. Trong hình này chỉ cần hiểu là **một giao tác đọc bị kéo dài**; các cột F-G trong manual biểu thị số chu kỳ chờ có thể kéo dài tùy ý, không phải giao tác đọc thứ hai.

4.7.2 Phân tích

Cột A — bắt đầu chu kỳ đọc trên cạnh lên `clk`.

Cột B — Bus đưa `address`, `byteenable_n`, `read_n` hợp lệ đến Slave.

Cột C — Bus giải mã địa chỉ và assert `chipselct` .

Cột D — Slave assert `waitrequest` trước cạnh clock kế tiếp vì chưa có `readdata` hợp lệ.

Cột E — Bus lấy mẫu thấy `waitrequest = 1` , vì vậy không capture `readdata` ; giao tác trở thành wait state.

Cột F-G — `waitrequest` còn được giữ mức 1 trong số chu kỳ tùy ý; Master/Bus phải giữ nguyên địa chỉ và tín hiệu điều khiển.

Cột H — Slave đưa `readdata` hợp lệ.

Cột I — Slave deassert `waitrequest` .

Cột J — Bus capture `readdata` ở cạnh lên kế tiếp, giao tác đọc kết thúc.

Ghi nhớ. Khác biệt mấu chốt với Hình 1 – Hình 3: số chu kỳ chờ không cố định trước, vì `waitrequest` phụ thuộc trạng thái thực của Slave tại thời điểm đó. Manual cũng nhấn mạnh Bus không có timeout; nếu Slave giữ `waitrequest` mãi, Master có thể bị treo.

4.8. Hình 5 — Slave Write cơ bản (zero wait state)

Hình 5 đối ứng với Hình 1 nhưng cho ghi. Master phát `write_n` , `writedata` thay vì `read_n` , `readdata` .

Cột A — chu kỳ nghỉ.

Cột B — Master phát `address` , `byteenable_n` , `writedata` ; `write_n` xuống 0. Các tín hiệu địa chỉ, dữ liệu và điều khiển bắt đầu hợp lệ ở phía Slave.

Cột C — Slave thấy `write_n = 0` và `chipselct = 1` , chốt `writedata` vào thanh ghi nội bộ ở cạnh lên clk.

Cột D — Master kết thúc giao tác, các tín hiệu trở về trạng thái không hoạt động.

Lưu ý so sánh Read và Write

Trong giao tác Read, Master **nhận** dữ liệu nên cần đợi Slave xuất `readdata` . Trong giao tác Write, Master **cấp** dữ liệu nên Slave chỉ việc chốt; vì vậy giao thức Write thường có ít chu kỳ chờ hơn ở dạng cơ bản.

4.9. Hình 6 — Slave Write với fixed wait state

Hình 6 thêm một fixed wait state giữa lúc Bus đưa địa chỉ/dữ liệu đến Slave và lúc Slave capture dữ liệu. Đây là wait state tĩnh do cấu hình Slave, không phải do `waitrequest`.

Cột A — chu kỳ nghỉ.

Cột B — Master phát `address`, `byteenable_n`, `writedata`; `write_n` xuống 0. Các tín hiệu địa chỉ, dữ liệu và điều khiển bắt đầu hợp lệ ở phía Slave.

Cột C — Bus giải mã địa chỉ và đặt `chipselct = 1`.

Cột D — fixed wait state duy nhất kết thúc ở cạnh lên; trong chu kỳ này `address`, `writedata`, `byteenable_n`, `write_n`, `chipselct` được giữ nguyên.

Cột E — Slave capture `writedata`, `address`, `byteenable_n`, `write_n`, `chipselct`; giao tác ghi kết thúc.

Cột F — Master kết thúc giao tác.

4.10. Hình 7 — Slave Write với waitrequest

Hình 7 thay fixed wait state bằng `waitrequest` cho ghi. Đây là kịch bản Slave có thời gian ghi không cố định, ví dụ cần đẩy dữ liệu vào FIFO mà FIFO hiện đang đầy. Tương tự Hình 4, các cột F-G là khoảng wait kéo dài tùy ý của **một giao tác ghi**, không phải giao tác thứ hai.

Cột A — bắt đầu giao tác ghi trên cạnh lên `clk`.

Cột B — Bus đưa `address`, `writedata`, `byteenable_n`, `write_n` hợp lệ đến Slave.

Cột C — Bus giải mã địa chỉ và assert `chipselct`.

Cột D — Slave assert `waitrequest` trước cạnh clock kế tiếp vì chưa capture được dữ liệu.

Cột E — Bus lấy mẫu thấy `waitrequest = 1`; chu kỳ trở thành wait state và mọi tín hiệu ghi phải giữ nguyên.

Cột F-G — `waitrequest` tiếp tục ở mức 1 trong số chu kỳ tùy ý.

Cột H — Slave cuối cùng capture `writedata`.

Cột I — Slave deassert `waitrequest`.

Cột J — giao tác ghi kết thúc ở cạnh lên kế tiếp.

Chu kỳ chờ động phía Write

Master luôn phải giữ `address`, `writedata`, `write_n`, `byteenable_n` cho đến khi `waitrequest` về 0. Nếu Master “buông” sớm, Slave có thể chót sai dữ liệu. Đây là điểm thường bị hỏi khi vấn đáp giao thức ghi có `waitrequest`.

4.11. Hình 8 — Master Read nhìn từ phía Master Port (zero wait state)

Hình 8 đặt vòng tròn nhận vào **Master Port** thay vì Slave Port: tài liệu nhìn các tín hiệu mà Master phát ra Bus và nhận vào từ Bus. Trong hình này, `waitrequest` không được assert nên giao tác đọc kết thúc trong một bus cycle. Điểm cần nhớ là ở góc nhìn Master, Master chỉ quan tâm: phát `address`, `byteenable_n`, `read_n`, chờ `waitrequest`, rồi capture `readdata`.

Cột A — chu kỳ đọc bắt đầu trên cạnh lên `clk`.

Cột B — Master assert `address`, `byteenable_n` và `read_n`.

Cột C — Bus trả `readdata` hợp lệ ngay trong bus cycle đầu; `waitrequest` không assert.

Cột D — Master capture `readdata` ở cạnh lên kế tiếp, deassert `address` và `read_n`; giao tác kết thúc.

Ghi nhớ. Dù `waitrequest` không assert trong Hình 8, tín hiệu này vẫn là một phần của giao thức Master Read. Nếu một Slave hoặc Bus chậm hơn, cùng Master đó sẽ phải chuyển sang hành vi của Hình 9.

4.12. Hình 9 — Master Read với waitrequest

Hình 9 là Master Read khi Avalon Bus Module assert `waitrequest`. Nếu `waitrequest` được giữ trong N bus cycle thì toàn bộ transfer kéo dài N + 1 bus cycle. Đây

là quy tắc vàng của Master: assert tín hiệu để bắt đầu transfer, sau đó giữ nguyên output cho đến khi `waitrequest` được deassert.

Cột A — bắt đầu giao tác đọc trên cạnh lên `clk`.

Cột B — Master assert `address`, `byteenable_n` và `read_n`.

Cột C — Avalon Bus Module assert `waitrequest` trước cạnh clock kế tiếp.

Cột D — Master thấy `waitrequest = 1`; bus cycle này trở thành wait state.

Cột E-F — khi `waitrequest` còn assert, Master giữ nguyên `address`, `byteenable_n`, `read_n`.

Cột G — `readdata` hợp lệ xuất hiện từ Bus.

Cột H — Bus deassert `waitrequest`.

Cột I — Master capture `readdata`, deassert output; giao tác kết thúc.

Ý nghĩa đối với DMA

Khi DMAC đóng vai trò Master để đọc dữ liệu, mỗi giao tác đọc vẫn tuân theo quy tắc Hình 8 hoặc Hình 9. Tổng thời gian truyền không chỉ phụ thuộc tần số clock mà còn phụ thuộc số chu kỳ `waitrequest` do Slave nguồn hoặc Bus gây ra.

4.13. Hình 10 — Master Write nhìn từ Master Port (zero wait state)

Hình 10 đối ứng với Hình 8 cho giao tác ghi. Master phát `address`, `byteenable_n`, `writedata`, `write_n`; nếu `waitrequest` không assert tại cạnh clock kế tiếp thì giao tác ghi kết thúc ngay trong một bus cycle.

Cột A — bắt đầu giao tác ghi trên cạnh lên `clk`.

Cột B — Master assert `address`, `byteenable_n`, `writedata`, `write_n`.

Cột C — `waitrequest` không assert tại cạnh lên; write transfer kết thúc. Master có thể bắt đầu transfer khác ở bus cycle kế tiếp.

4.14. Hình 11 — Master Write với hai chu kỳ waitrequest

Hình 11 minh họa Master Write khi Avalon Bus Module assert `waitrequest` trong hai bus cycle. Toàn bộ write transfer vì vậy kéo dài ba bus cycle. Đây là tình huống Slave chưa capture được dữ liệu ngay, ví dụ FIFO đầy hoặc bộ nhớ đang bận arbitration với Master khác.

Cột A — bắt đầu giao tác ghi trên cạnh lên `clk`.

Cột B — Master assert `address`, `byteenable_n`, `writedata`, `write_n`.

Cột C — `waitrequest = 1`; đây là wait state thứ nhất, Master giữ nguyên toàn bộ output.

Cột D — `waitrequest` vẫn bằng 1; đây là wait state thứ hai, Master tiếp tục giữ nguyên output.

Cột E — Bus deassert `waitrequest`.

Cột F — tại cạnh lên kế tiếp, vì `waitrequest` không assert, Master deassert output và write transfer kết thúc.

Sự khác biệt giữa Hình 10 và Hình 11

Hình 10 là Master Write không có wait state; Hình 11 là Master Write có hai wait states do `waitrequest`. Nguyên tắc thiết kế Master không đổi: giữ `address`, `byteenable_n`, `writedata`, `write_n` cho đến cạnh clock sau khi `waitrequest` được deassert.

4.15. Tóm tắt khác biệt giữa các hình

Hình	Loại giao tác	Quản lý chờ	Điểm trọng tâm
Hình 1	Slave Read	Zero wait state	Giao tác đọc cơ bản; Slave xuất <code>readdata</code> ngay sau khi <code>chipselct</code> lên.
Hình 2	Slave Read	1 fixed wait state	Bus chèn 1 chu kỳ chờ tĩnh trước khi <code>readdata</code> hợp lệ.
Hình 3	Slave Read	2 fixed wait states	Mở rộng của Hình 2 với 2 chu kỳ chờ tĩnh.
Hình 4	Slave Read	<code>waitrequest</code>	Slave chủ động kéo dài một giao tác đọc; Bus capture <code>readdata</code> sau khi <code>waitrequest</code> hạ.
Hình 5	Slave Write	Zero wait state	Giao tác ghi cơ bản; Slave chốt <code>writedata</code> .
Hình 6	Slave Write	1 fixed wait state	Bus chèn một chu kỳ chờ tĩnh trước khi Slave capture <code>writedata</code> .
Hình 7	Slave Write	<code>waitrequest</code>	Slave kéo dài một giao tác ghi; Master giữ <code>writedata</code> đến khi <code>waitrequest</code> hạ.
Hình 8	Master Read	Zero wait state	Góc nhìn Master Port; <code>waitrequest</code> không assert, Master capture <code>readdata</code> sau một bus cycle.
Hình 9	Master Read	<code>waitrequest</code>	Góc nhìn Master Port; nếu <code>waitrequest</code> giữ N chu kỳ thì transfer kéo dài N + 1 chu kỳ.
Hình 10	Master Write	Zero wait state	Góc nhìn Master Port; <code>waitrequest</code> không assert nên ghi xong trong một bus cycle.
Hình 11	Master Write	2 wait states	Góc nhìn Master Port; Master giữ tín hiệu qua hai chu kỳ <code>waitrequest</code> .

Bảng 26 : Tóm tắt nhanh đặc điểm Hình 1 – Hình 11

4.16. Cặp so sánh hay bị hỏi

Ngoài việc phân tích từng hình, cần chuẩn bị các cặp so sánh vì câu hỏi vẫn đáp thường ở dạng “Hình này khác hình kia chỗ nào?”.

Cặp so sánh	Điểm khác biệt cần trả lời
Hình 1 và Hình 5	Cùng zero wait state nhưng Hình 1 là Read nên dữ liệu nằm trên <code>readdata</code> ; Hình 5 là Write nên dữ liệu nằm trên <code>writedata</code> .
Hình 2 và Hình 3	Cùng là Read có fixed wait state, khác số chu kỳ chờ tĩnh: một chu kỳ so với hai chu kỳ.
Hình 3 và Hình 4	Cùng làm Master phải đợi, nhưng Hình 3 đợi theo cấu hình cố định, Hình 4 đợi theo <code>waitrequest</code> động từ Slave.
Hình 4 và Hình 7	Cùng dùng <code>waitrequest</code> ở phía Slave; Hình 4 là đọc nên chờ <code>readdata</code> , Hình 7 là ghi nên giữ <code>writedata</code> đến khi Slave capture được.
Hình 8 và Hình 9	Cùng là Master Read; Hình 8 không có wait state, Hình 9 có <code>waitrequest</code> nên Master giữ output lâu hơn.
Hình 10 và Hình 11	Cùng là Master Write; Hình 10 không có wait state, Hình 11 có hai chu kỳ <code>waitrequest</code> .

Bảng 27 : Các cặp hình nên so sánh được bằng lời

4.17. Mẫu trả lời khi giảng viên chỉ vào một cột thời gian

Khi bị hỏi “tại cột C đang xảy ra gì?”, câu trả lời nên có ba ý: trạng thái tín hiệu, vai trò của thành phần, và hệ quả sang chu kỳ sau. Ví dụ với một cột đang có `waitrequest = 1` , không nên chỉ nói “đang wait”; nên nói đầy đủ hơn:

Mẫu câu

Ở cột này, Slave hoặc Bus đang kéo `waitrequest = 1` , nghĩa là giao tác chưa thể hoàn tất. Vì vậy Master phải giữ nguyên `address` , `read_n/write_n` , `byteenable_n` và nếu là ghi thì giữ cả `writedata` . Sang chu kỳ sau, chỉ khi `waitrequest` về 0 thì dữ liệu đọc mới được lấy mẫu hoặc dữ liệu ghi mới được chốt.

Nếu cột đó không có `waitrequest` nhưng vẫn là chu kỳ chờ, hãy nói đây là fixed wait state: Bus giữ tín hiệu theo số chu kỳ đã khai báo trong cấu hình Slave, không phải do Slave kéo `waitrequest` động.

4.18. Câu hỏi ôn tập gợi ý

Ghi nhớ.

1. So sánh giao tác đọc trong Hình 1, Hình 2 và Hình 3. Khác biệt nằm ở đâu? Bus dựa vào đâu để chèn chu kỳ chờ tĩnh?
2. So sánh Hình 4 và Hình 7: cả hai đều dùng `waitrequest`, nhưng một bên là đọc và một bên là ghi. Tín hiệu nào do ai phát, ai phải giữ?
3. Tại sao Hình 8 và Hình 9 lại nhìn từ Master Port? So sánh trường hợp không có và có `waitrequest`.
4. Trong Hình 11, vì sao Master phải giữ nguyên `address`, `writedata` qua hai chu kỳ `waitrequest`? Điều gì sẽ xảy ra nếu Master rút sớm?
5. Cho một giản đồ ngẫu nhiên không có nhãn, hãy chỉ ra đó là Read hay Write, có dùng `waitrequest` hay không, và đếm số chu kỳ chờ.
6. Tín hiệu `chipselct` xuất hiện sau bước giải mã nào của Bus? Nếu `chipselct = 0` thì Slave có được phép chốt `writedata` không?
7. Vì sao không được hiểu cột F-G trong Hình 4 hoặc Hình 7 là giao tác mới? Dấu hiệu nào cho thấy vẫn là một giao tác đang bị kéo dài?

CHƯƠNG 5

TỔNG KẾT VÀ CHIẾN LƯỢC ÔN TẬP

5.1. Bản đồ kiến thức theo bốn nội dung

Bốn nội dung đề cương không tách rời mà liên kết chặt chẽ. Khái niệm Master, Bus, Slave (Chương 1) là nền tảng để hiểu mọi project; quy trình thiết kế SoPC (Chương 2) là khung làm việc chung cho cả bốn project (Chương 3); và các giản đồ thời gian (Chương 4) chính là hình ảnh hóa cụ thể của những gì đã trừu tượng hóa ở Chương 1. Khi vấn đáp, sinh viên nên trả lời theo trình tự “khái niệm → quy trình → ví dụ project → giản đồ” để câu trả lời có nền lý thuyết và minh họa cụ thể.

Nội dung	Cốt lõi	Liên kết với phần khác
Master/Bus/Slave	Vai trò, tín hiệu, chu kỳ chờ.	Là nền tảng để đọc giản đồ thời gian.
Quy trình SoPC	Sáu bước; biên giới phần cứng – phần mềm là <code>system.h</code> .	Áp dụng cho cả bốn project; mọi đổi Qsys đều cần sinh BSP lại.
Bốn project	PIO HEX, Custom IP HEX, Timer/IRQ, DMAC.	Mỗi project minh họa một khía cạnh của Bus: Slave có sẵn, Slave do mình viết, Slave + IRQ, Master + Slave kết hợp.
Hình 1 – Hình 11	Read/Write, fixed wait, waitrequest, Master vs. Slave Port.	Là minh họa thời gian của các tín hiệu đã giới thiệu ở Chương 1.

Bảng 28 : Bản đồ kiến thức tổng quát

5.2. Lộ trình ôn tập gợi ý

Để ôn tập hiệu quả với thời gian có hạn, có thể chia làm bốn buổi ngắn:

1. **Buổi 1 — Khái niệm và tín hiệu:** đọc lại Chương 1 và Chương 4 song song. Mục tiêu là gọi tên đúng các tín hiệu Avalon và biết ai phát ai nhận.

2. **Buổi 2 — Quy trình thiết kế:** đọc Chương 2; thử kể lại bằng lời sáu bước, lưu ý điểm “biên giới” `system.h`.
3. **Buổi 3 — Bôn project:** đọc Chương 3, vẽ lại sơ đồ khối phần cứng cho từng project trên giấy nháp; viết lại đoạn pseudo-code cho phần phần mềm.
4. **Buổi 4 — Giản đồ thời gian:** lấy đề cương gốc của giảng viên, đối chiếu từng Hình 1 – Hình 11 với phân phân tích trong Chương 4, tự đánh dấu cột thời gian nào ứng với “address valid”, “readdata valid”, “waitrequest = 1”.

Mẹo trả lời câu hỏi giản đồ

Khi giảng viên đưa một giản đồ, đừng vội bắt đầu giải thích từ cột A. Đầu tiên xác định **Read hay Write, có waitrequest hay không, bao nhiêu giao tác**. Sau khi định danh đúng, nội dung phân tích từng cột sẽ đi theo khung quen thuộc đã có trong tài liệu.

5.3. Những lỗi thường gặp khi vấn đáp

- **Nhầm vai trò tín hiệu:** nói `chipselect` do Master phát thay vì do Bus phát. Đây là lỗi nhỏ về thuật ngữ nhưng dễ bị trừ điểm.
- **Quên tín hiệu được giữ trong khi đợi:** khi `waitrequest = 1`, Master **phải giữ** `address`, `read_n / write_n`, `byteenable_n` và nếu là ghi thì giữ cả `writedata`. Quên điểm này dẫn đến hiểu sai giao thức.
- **Lẫn lộn fixed wait state với waitrequest:** hai cơ chế này khác nhau về bản chất. Fixed wait state do Bus chèn theo cấu hình tĩnh; `waitrequest` do Slave phát theo trạng thái thực.
- **Quên sinh BSP sau khi đổi Qsys:** lỗi thực hành thường dẫn đến “code đúng nhưng kit không phản ứng”.
- **Nhầm DMAC chỉ là Slave:** thực ra DMAC vừa là Slave (cho CPU cấu hình) vừa là Master (tự sinh giao tác đọc/ghi).
- **Quên xóa cờ TO của Timer trong ISR (Prj3):** dẫn đến ngắt liên tục, treo CPU.

5.4. Ngân hàng câu hỏi ngắn và ý trả lời

Câu hỏi	Ý trả lời cần có
Master khác Slave ở đâu?	Master chủ động phát địa chỉ và yêu cầu đọc/ghi; Slave thụ động phản hồi khi được Bus chọn bằng <code>chipsselect</code> .
Bus làm gì ngoài nối dây?	Giải mã địa chỉ, định tuyến tín hiệu, sinh <code>chipsselect</code> , xử lý arbitration khi nhiều Master và chèn chu kỳ chờ.
Fixed wait state khác <code>waitrequest</code> ở đâu?	Fixed wait state là số chu kỳ chờ cấu hình trước; <code>waitrequest</code> là tín hiệu động do Slave/interconnect kéo lên khi chưa sẵn sàng.
<code>byteenable_n</code> dùng làm gì?	Chỉ ra byte nào trong word có hiệu lực. Vì active-low nên bit 0 nghĩa là byte tương ứng được chọn.
Tại sao phải sinh lại BSP?	Vì BSP sinh <code>system.h</code> ; nếu Qsys đổi địa chỉ hoặc IRQ mà BSP cũ thì chương trình C dùng thông tin sai.
Prj2 chứng minh kỹ năng gì?	Biết tự viết IP Avalon-MM Slave write-only, khai báo <code>chipsselect/address/write/writedata</code> trong Component Editor, export conduit <code>hex0 ... hex5</code> , và ghi IP bằng <code>IOWR(HEX_0_BASE, index, digit)</code> .
Prj3 hơn Prj1 ở đâu?	Timer phần cứng tạo mốc 1 giây ổn định hơn, CPU không bị busy-wait và có thể xử lý việc khác giữa hai ngắt.
DMAC vừa Master vừa Slave nghĩa là gì?	CPU cấu hình DMAC qua vai trò Slave; sau đó DMAC tự phát giao tác đọc/ghi qua vai trò Master để truyền dữ liệu.
Khi nào cần <code>volatile</code> trong C?	Khi biến hoặc địa chỉ có thể thay đổi ngoài luồng lệnh bình thường, ví dụ thanh ghi phần cứng hoặc biến được ISR cập nhật.
Nếu <code>waitrequest = 1</code> , Master	Giữ nguyên địa chỉ, điều khiển, <code>byteenable</code> và dữ

phải làm gì? **Bảng 29** : Câu hỏi ngắn liên quan đến khi `waitrequest = 0`.

5.5. Khung trả lời 2 phút khi bị hỏi một project

Khi giảng viên yêu cầu trình bày một project bất kỳ, có thể trả lời theo khung 2 phút sau:

1. **Mở đầu:** nêu mục tiêu project và IP chính được dùng.
2. **Phần cứng:** chỉ ra CPU Nios II là Master, các IP nào là Slave, có IRQ hoặc DMAC Master phụ hay không.
3. **Memory map:** nói các Slave có base address riêng; phần mềm truy cập qua `system.h`.
4. **Phần mềm:** tóm tắt thuật toán C, vòng lặp chính, ISR hoặc DMA nếu có.
5. **Điểm cần nhớ:** nêu ưu/nhược điểm hoặc lỗi hay gặp của project đó.

Ví dụ câu kết

Như vậy project này không chỉ là làm đồng hồ chạy, mà là minh họa một kỹ thuật thiết kế hệ thống nhúng: Prj1 là PIO HEX và polling, Prj2 là custom IP HEX, Prj3 là Timer interrupt, Prj4 là DMA có thêm một Master trên Bus.

5.6. Checklist cuối trước khi thi

Ghi nhớ. Trước khi vào thi, nên tự kiểm tra rằng mình làm được các việc sau:

1. Vẽ lại được sơ đồ Master - Bus - Slave và ghi tên các tín hiệu chính.
2. Giải thích được read/write, fixed wait state và `waitrequest` mà không nhìn tài liệu.
3. Kể lại được sáu bước thiết kế SoPC từ Quartus đến nạp `.sof` và `.elf`.
4. Với mỗi project, chỉ ra được Master, Slave, bảng địa chỉ, thuật toán phần mềm và điểm trọng tâm.
5. Nhìn một giản đồ thời gian bất kỳ, xác định được Read hay Write, có bao nhiêu chu kỳ chờ và tín hiệu nào phải giữ.

5.7. Lời kết

Đề cương ôn tập này được biên soạn trên tinh thần “hiểu rõ vài điểm cốt lõi, thay vì học vẹt nhiều chi tiết rời rạc”. Sinh viên chỉ cần nắm vững vai trò của Master, Bus, Slave và bốn cơ chế quản lý chu kỳ chờ là đã đủ để phân tích bất kỳ giản đồ Avalon nào, kể cả giản đồ chưa từng thấy. Khi vấn đáp các project, hãy nhớ trình bày theo bốn mục cố định

(sơ đồ khối, bảng địa chỉ, thuật toán, ý chính phần mềm) để câu trả lời có cấu trúc rõ ràng và bao quát.

Chúc sinh viên ôn tập hiệu quả và đạt kết quả tốt trong kỳ thi.

TÀI LIỆU THAM KHẢO

- [1] Altera Corporation, “Avalon Interface Specifications”. 2014.
- [2] Altera Corporation, “Avalon Bus Specification Reference Manual”. 2003.
- [3] FETEL - HCMUS, “Buổi 3 - Làm quen với HEX trên DE10 Standard và Buổi 4 - Thiết kế IP cho LED 7 đoạn”. 2026.
- [4] Intel Corporation, “Qsys System Design Tutorial and Platform Designer User Guide”. 2018.
- [5] Intel Corporation, “Nios II Processor Reference Handbook”. 2020.
- [6] Intel Corporation, “Nios II Software Developer's Handbook”. 2020.
- [7] Intel Corporation, “Embedded Peripherals IP User Guide — PIO Core”. 2019.
- [8] Intel Corporation, “Embedded Peripherals IP User Guide — Interval Timer Core”. 2019.
- [9] Intel Corporation, “Embedded Peripherals IP User Guide — DMA Controller Core”. 2019.
- [10] Giảng viên học phần, “Đề cương ôn tập học phần Hệ thống nhúng”. 2026.